FLEXISPACE
"WORKOUTSIDETHEBOX"

FlexiSpace

Multi-Boardroom Booking System

Task 1 Documentation

Team Number	16
Team Name	Codeunit
Client	FlexiSpace

Team members

Full Name	Student Number	Group
Khensani Maluleke	ST10451309	2
Khumo-Thato Chabeli	ST10448834	3
Orearabetse Riba	ST10446648	2
Denzel Tinotenda Pawandiwa	ST10457186	2
Tinotenda Nyakatsari	ST10437488	2

Table of Contents

Table of Figures iv
9.1.4 Introduction 1
9.1.5 Requirements 3
9.1.6 User Roles 5
9.1.6.1 FlexiSpace User roles 6
9.1.6.2 Permission Matrix 7
9.1.6.3 Role Descriptions 8
9.1.7 User Stories and UX Journey map 9
9.1.7.1 Work Breakdown Structure 25
9.1.8 Non-functional Requirements 27
9.1.9 Analysis Artifacts 29
9.1.9.1 Domain Modelling 29
Domain Class Diagram 30
Core Domain Classes Explained 30
Key Relationships 31
9.1.9.2 Design Artifacts 31
Design Class Diagram 32
Implementation Classes 32
Keeping Things Clean 33
9.1.10 Implementation 34
9.1.10.1 Sequence Diagrams 34
User Books a Boardroom (Success) 34
Booking Conflict Detection 35
9.1.10.2 State Diagram 36
9.1.10.3Deployment Strategy 37
Deployment Plan 37
Rollout Plan 38
Technical Stack 38
9.1.11 Data Schema 39
9.1.11.1 Technology Choice: 39
9.1.11.2 ERD of FlexiSpace Database 40

ERD Explanation 41
9.1.12 Architecture Artifacts 43
9.1.12.1 Design Patterns 43
UML Diagram Example of Service Layer and Dependency Injection 44
9.1.12.2 Architecture Patterns 45
Architecture Pattern: Layered Architecture 45
Why we chose this architecture. 45
System Architecture Diagram: Layered Architecture 46
9.1.12.3 Cloud Architecture 47
Why We Chose This Architecture 48
Network flow Explanation: 48
Architecture Diagram of FlexiSpace 50
9.1.13 Security 51
9.1.13.1 Microsoft Entra ID Authentication 51
9.1.13.2 Authorisation and RBAC (Role based Access control) 52
9.1.13.3 Secure Token Storage on the Mobile Device 52
9.1.13.4 Secure Storage on the Mobile Application 53
9.1.13.6 API as the Security Boundary 55
9.1.13.7 Server-Side Input Validation 56
9.1.13.8 Secrets Management 57
9.1.13.9 Cross-Site Scripting (XXS) 57
9.1.14 DevOps 59
9.1.14. 1 Repository Structure 59
9.1.14.2 Branching Strategy 59
9.1.14.3 Pull Request Process 59
9.1.14.4 Commit Messages 60
9.14.5 Project Management (GitHub Projects) 60
9.1.14.6 CI/CD Pipeline (Plan) 61
9.1.14.7 Attendance Tracking 61
9.1.14.8 README Maintenance 61
9.1.15 Running Costs 62
9.1.15.1 Purpose 62

9.1.15.2 Running Costs vs. Build Costs 62
9.1.15.3 Cost Components Explained 62
9.1.15.4 Two Cost Scenarios 64
9.1.16 Change Management 66
9.1.16.1 Getting the Organisation On Board 66
9.1.16.2 Getting Users On Board 66
Adoption Plan 66
Training 66
9.1.16.3 Handing Over to the Client 66
9.1.16.4 Support Plan 67
9.1.16.5 Transitioning from Old to New 67
9.1.17 Appendices 68
References 69

Table of Figures

Figure 1: Work Breakdown Structure 25
Figure 2: Domain Class Diagram 30
Figure 3: Design Class Diagram 32
Figure 4: User Books a Boardroom (Success) 34
Figure 5: Booking Conflict Detection 35
Figure 6: Booking Lifecycle 36
Figure 7: Deployment Architecture 37
Figure 8: FlexiSpace ERD 40
Figure 9:UML diagram of Boardroom implementation. 44
Figure 10: FlexiSpace Layered Architecture 46
Figure 11:Cloud Architecture diagram 50

9.1.4 Introduction

1. Purpose

This document serves as the Software Requirements Specification (SRS) for the FlexiSpace Boardroom Booking System. It provides a detailed description of the system's functional and non-functional requirements, user roles, system architecture, and design constraints. The document is intended to align all stakeholders on the project scope and ensure the development team delivers a solution that meets the client's needs.

2. Document Conventions

The following conventions are used throughout this document:

- **Must / Shall:** Indicates a mandatory requirement that must be implemented.
- **Should:** Indicates a recommended requirement that is desirable but not mandatory.
- **Will:** Indicates a statement of fact about future system behavior.
- **Req ID:** Unique identifiers follow the format FR-[CATEGORY]-### for functional requirements and NFR-[CATEGORY]-### for non-functional requirements.

3. Project Scope

The FlexiSpace Boardroom Booking System is a centralized platform designed to manage boardroom reservations across three FlexiSpace locations: Centurion, Houghton, and Eagle Canyon. The system will replace the current manual booking process (phone/email requests and manual Outlook calendar checks) with an automated solution that provides real-time availability, two-way Outlook calendar synchronization, and automatic conflict detection.

The system will serve internal staff, Centre Managers, and Administrators, with future scope for external client self-service booking. The solution will be delivered as a web application (React + Fluent UI) and a mobile application (.NET MAUI), both consuming a common .NET 8 Web API backend hosted on Microsoft Azure with Microsoft Entra ID authentication.

4. Intended Audience

This document is intended for:

- **Development Team:** To understand system requirements and guide implementation
- **Project Manager:** To track progress against requirements

- **Client Stakeholders (FlexiSpace):** To validate that the proposed solution addresses their business needs
- **Quality Assurance:** To verify the system meets specified requirements during testing

9.1.5 Requirements

A. User Management & Authentication

Req ID	What it does	Priority
FR-AUTH-001	Log in using existing FlexiSpace Microsoft 365 credentials	High
FR-AUTH-002	Only FlexiSpace staff can log in - no outsiders	High
FR-AUTH-003	Auto log-out after inactivity	Medium
FR-USER-001	Three user types: Administrator, Centre Manager, Staff	High
FR-USER-002	Admins can add, edit, and remove users	High
FR-USER-003	Admins can change someone's role (make them a manager, etc.)	High
FR-USER-004	Admins can assign Centre Managers to specific locations	High
FR-USER-005	Admins can see all bookings everywhere	Medium

B. Location & Boardroom Management

Req ID	What it does	Priority
FR-LOC-001	Support Centurion, Houghton, and Eagle Canyon	High
FR-LOC-002	Admins can add or remove locations	High
FR-LOC-003	Admins can add or remove boardrooms within locations	High
FR-LOC-004	Each boardroom tracks: name, capacity, equipment	High
FR-LOC-005	Admins can block rooms for maintenance	Medium
FR-LOC-006	Link each boardroom to the right Outlook calendar(s)	High

C. Booking Management

Req ID	What it does	Priority
FR-BOOK-001	Staff and Centre Managers can create bookings	High
FR-BOOK-002	Booking needs: date, time, location, boardroom, purpose	High
FR-BOOK-003	Optional extras: attendee count, catering, equipment	Medium
FR-BOOK-004	Users can view their own bookings	High
FR-BOOK-005	Staff can edit their future bookings	High
FR-BOOK-006	Staff can cancel their future bookings	High
FR-BOOK-007	Centre Managers see all bookings for their location(s)	High
FR-BOOK-008	Centre Managers can edit or cancel any booking at their site	High
FR-BOOK-009	Admins can manage any booking anywhere	High
FR-BOOK-010	Soft delete - bookings get archived, not permanently wiped	High
FR-BOOK-011	Every booking gets a unique ID	High
FR-BOOK-012	Centre Managers can block off rooms for maintenance	Medium

D. Conflict Detection & Availability

Req ID	What it does	Priority
FR-CON-001	Check if a room is free before confirming a booking	High
FR-CON-002	Check BOTH the database AND Outlook calendar	High
FR-CON-003	Absolutely no double-bookings allowed	High
FR-CON-004	If conflict found, reject booking and explain why	High
FR-CON-005	Suggest alternative times when a room is taken	Medium
FR-CON-006	Show real-time availability (Free, Busy, Maintenance)	High

E. Outlook Calendar Integration

Req ID	What it does	Priority
FR-OUT-001	Connect to Outlook via Microsoft Graph API	High
FR-OUT-002	Centurion bookings sync to centurion@flexispacenet.za	High
FR-OUT-003	Houghton bookings sync to rebecca@flexispacenet.za	High
FR-OUT-004	Eagle Canyon syncs to both antoinette@... and lesego@...	High
FR-OUT-005	Create Outlook event when booking is made	High
FR-OUT-006	Update Outlook event when booking changes	High
FR-OUT-007	Delete Outlook event when booking is cancelled	High
FR-OUT-008	If someone changes something in Outlook, the system notices and updates (two-way sync)	High
FR-OUT-009	Use webhooks for instant sync where possible	Medium
FR-OUT-010	Fallback: sync every 2 minutes if webhooks fail	Medium

F. Notifications

Req ID	What it does	Priority
FR-NOT-001	Email confirmation when you book a room	High
FR-NOT-002	Email when a booking is changed	High
FR-NOT-003	Email when a booking is cancelled	High
FR-NOT-004	Reminder 24 hours before your booking	High
FR-NOT-005	Reminder 1 hour before your booking	High
FR-NOT-006	In-app notifications on mobile for confirmations and reminders	Medium
FR-NOT-007	Email the Centre Manager when someone books their room	Medium

G. Search & Filtering

Req ID	What it does	Priority
FR-SEA-001	Search for rooms by location	High
FR-SEA-002	Filter by capacity	Medium
FR-SEA-003	Filter by equipment (TV, projector, whiteboard)	Medium
FR-SEA-004	Search bookings by date range	High
FR-SEA-005	Pagination on all lists (so we're not loading 500 bookings at once)	High
FR-SEA-006	Sort bookings by date, time, location	Medium

H. Reporting & Analytics

Req ID	What it does	Priority
FR-REP-001	Daily booking reports	High
FR-REP-002	Monthly usage reports	High
FR-REP-003	Show occupancy percentages for each room	High
FR-REP-004	Identify most and least used boardrooms	High
FR-REP-005	Report on cancelled bookings	Medium
FR-REP-006	Export reports as PDF or CSV	Medium
FR-REP-007	Dashboard showing: today's bookings, current availability, upcoming meetings	High

I. Audit Trail

Req ID	What it does	Priority
FR-AUD-001	Log who created each booking and when	High
FR-AUD-002	Log who changed a booking and what changed	High
FR-AUD-003	Log who cancelled a booking	High
FR-AUD-004	Log who logged in and when	High
FR-AUD-005	Admins can view the audit trail	Medium

9.1.6 User Roles

Flexispace is comprised of four user roles- Staff, Client, Centre Manager, and Administrator - each with a different scope of access to bookings, rooms, and system configuration. This document sets out the permission matrix, role descriptions, demo

accounts for testing, key business rules, and the full set of user stories driving the system's functionality.

9.1.6.1 FlexiSpace User roles

Role	Description	Permissions/Scope
Staff	Any employee at Centurion, Houghton, or Eagle Canyon with a Microsoft 365 account. Logs in via Entra ID and uses Flexispace to book a room for a meeting. This is the default role for every internal user.	View live room availability at all three locations. Book a room (request goes in as pending until approved). Cancel or edit their own bookings. View their own booking history. Receive notifications. Cannot approve bookings, block rooms, or see other users' bookings.
Client	An external guest to the business (not a Flexispace/Microsoft 365 account holder). Logs in with an email and password issued to them, not Entra ID.	Book a room via a guided form (request goes in as Pending until approved). Cancel/Edit their own bookings. Receive notifications. Cannot view the live room -by-room availability grid – booking is self-service rather than browse-and-pick. Online payment for bookings is planned for phase 2, not this submission.
Centre Manager	Manager is responsible for one specific centre's boardrooms.	Everything Staff can do at their centre, plus: view every booking (not just their own) for their centre. Approve or decline Pending bookings. Edit or Cancel any bookings at their centre. Block a room for a period (e.g. maintenance, cleaning). Scope is limited to their own centre only.
Administrator	System-wide Administrator, not tied to one centre.	Everything a Centre Manager can do, but across all three centres rather than one. On the website only, additionally: add/remove rooms. Configure locations. Manage user accounts and roles. View reporting/usage statistics and export PDF/CSV. These admin-configuration screens are intentionally not built into the mobile app (see note below).

All four roles are enforced server-side on every API endpoint, never trusted from the front end alone. A user's role is checked before any restricted action (approving a booking, blocking a room, deleting a room) is allowed to proceed, regardless of which client – Website or mobile – sent the request.

9.1.6.2 Permission Matrix

Capability	Staff	Client	Centre Manager	Administrator
Book rooms	✓	✓	✓ **(own site)**	✓
Cancel own bookings	✓	✓	✓	✓
View live availability	✓	✗	✓ **(own site)**	✓ **(all sites)**
Approve / decline bookings	✗	✗	✓ **(own site)**	✓ **(all sites)**
Edit any booking	✗	✗	✓	✓
Cancel any booking	✗	✗	✓	✓
Block rooms	✗	✗	✓ **(own site)**	✓ **(all sites)**
View all bookings in scope	✗	✗	✓ **(location)**	✓ **(all)**
Access Manage console	✗	✗	✓	✓
Add / remove rooms	✗	✗	✗	✓
Configure locations	✗	✗	✗	✓
Manage users	✗	✗	✗	✓
View usage reports	✗	✗	✗	✓
Online payment (Phase 2)	✗	✓ **(planned)**	✗	✗

9.1.6.3 Role Descriptions

- Administrator: add/remove rooms, configure locations, manage users, view all bookings, view usage reports.
- Centre Manager: view, approve, block, edit, and cancel bookings for their own centre.
- Staff: book rooms, cancel their own bookings, view live availability.
- Client: self-service booking, receive confirmations, with online payments planned for Phase 2.

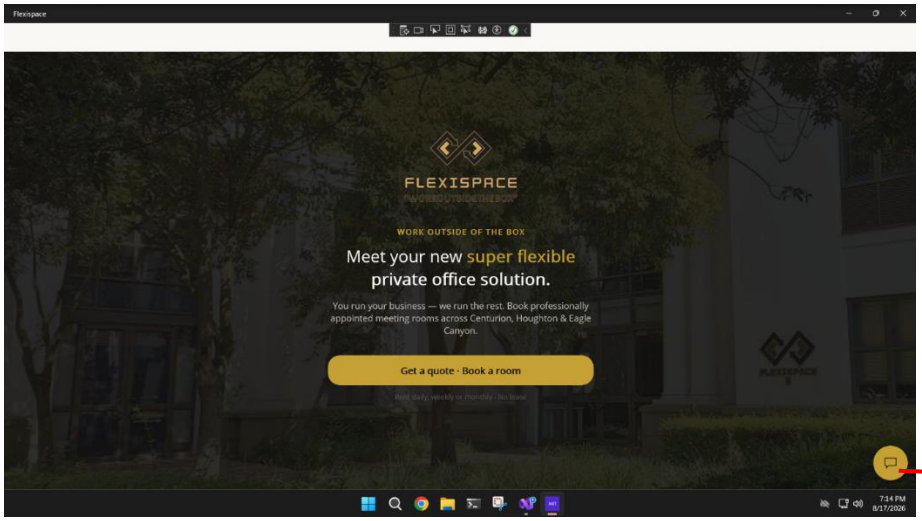
9.1.7 User Stories and UX Journey map

The user stories below follow the standard “As a [role], I want to [action], so that [benefit]” format, grouped by feature area: authentication, booking, the approval/room-blocking workflow, calendar/notifications, the AI feature, admin functions, and security. The UX Journey map is added below each User story table

All Users

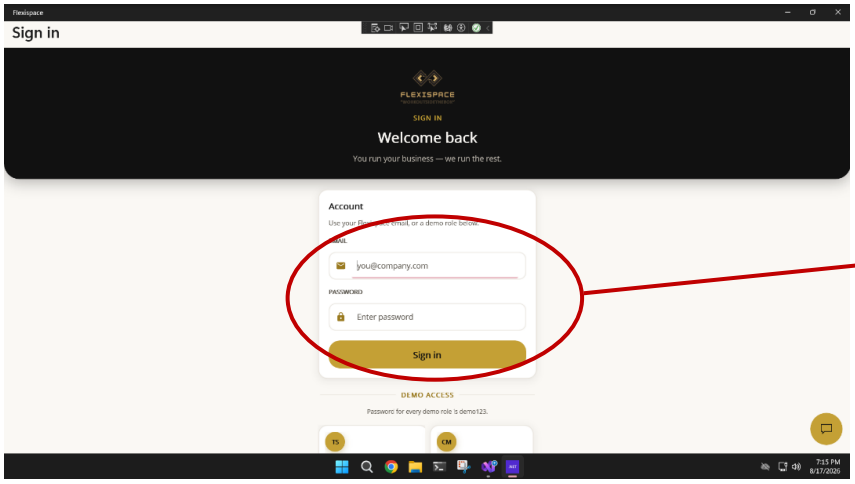
#	User Story
1	As a signed-in user, I want to sign in with email and password so that I can access the booking system.
2	As a signed-in user, I want a dashboard with today's bookings and location info so that I can quickly see my schedule.
3	As a signed-in user, I want alerts for confirmations, reminders, and cancellations so that I stay informed.
4	As a signed-in user, I want to view my booking history so that I can track past and upcoming reservations.
5	As a signed-in user, I want to browse all Flexispace locations so that I can choose where to meet.

Launch page

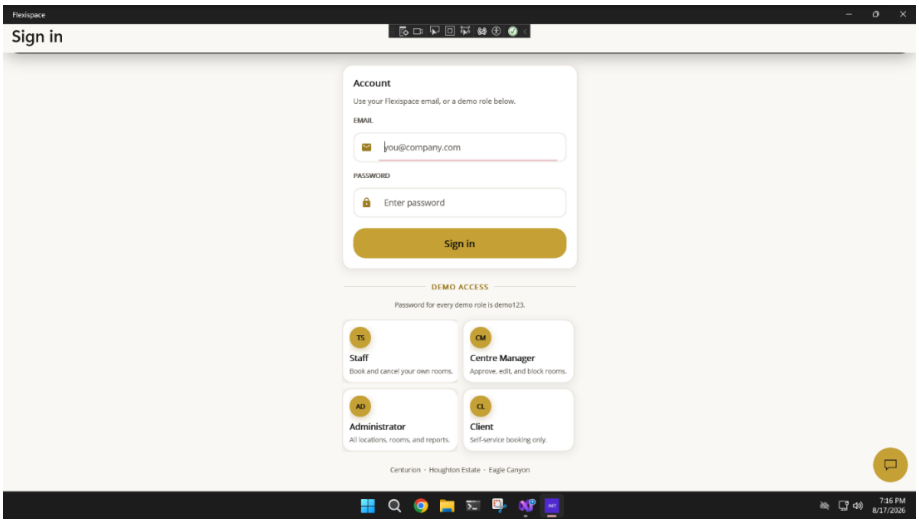


AI Chatbot

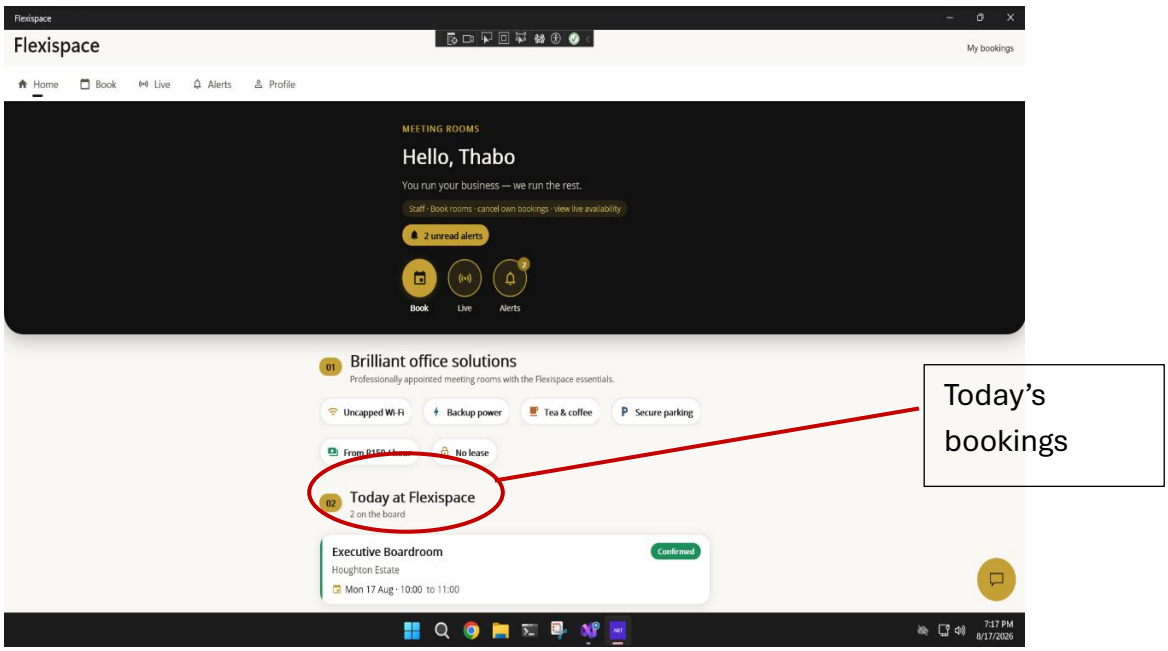
1. **All users can Sign in**



Email & Password

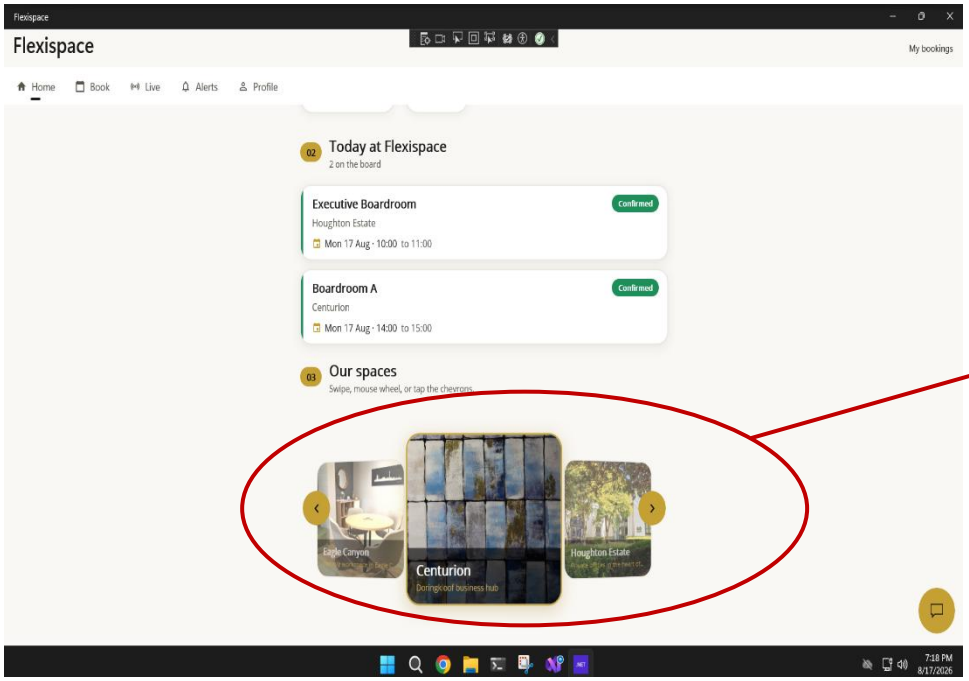


All Users can view live bookings



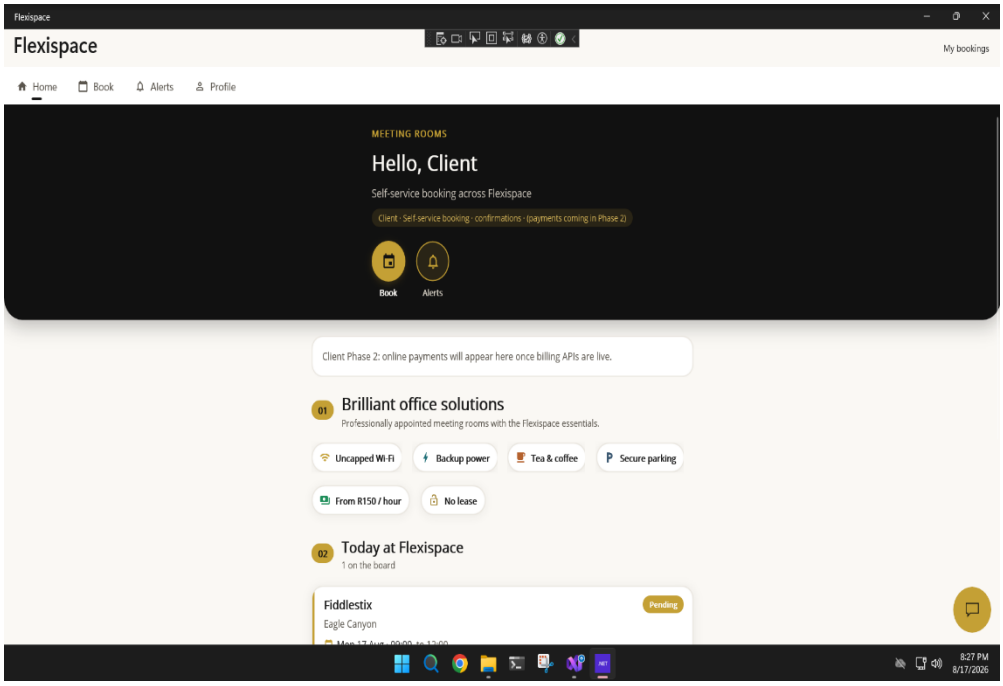
Today's bookings

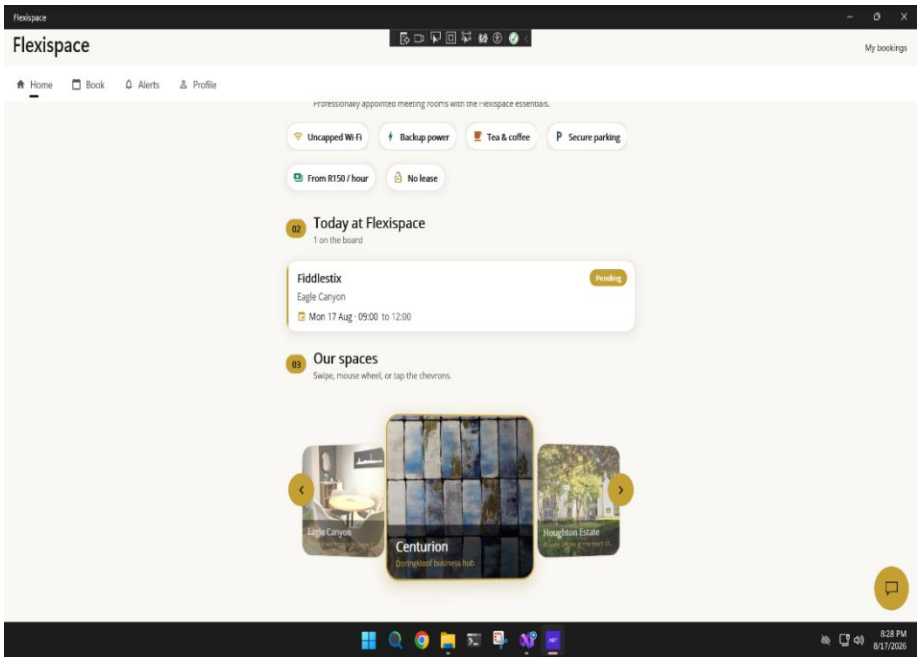
All User can View Locations



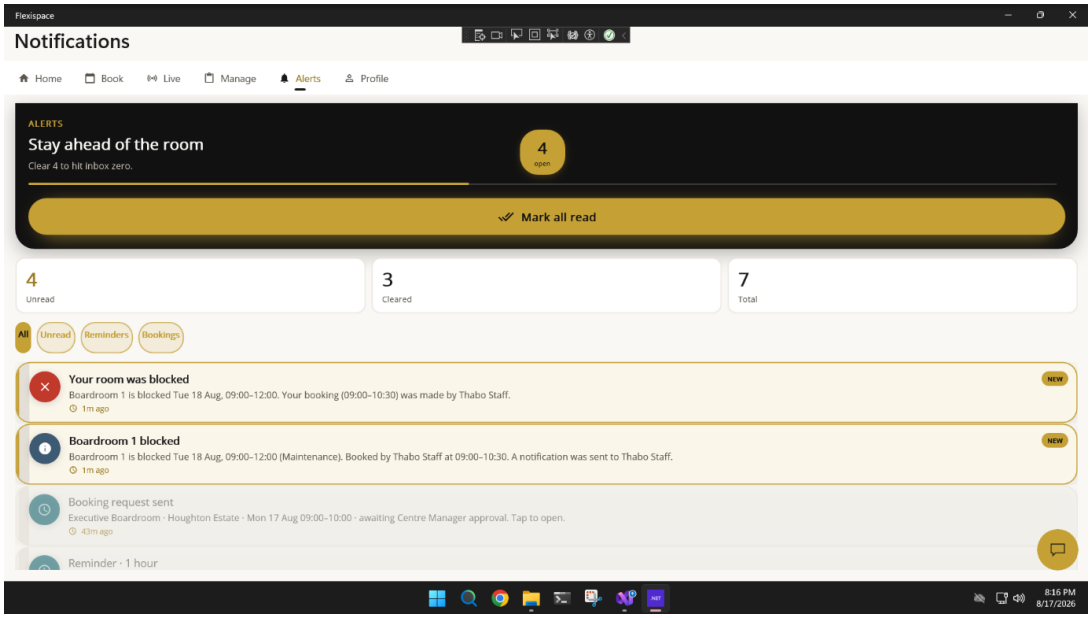
Location info

2.1 Dashboards (Client)

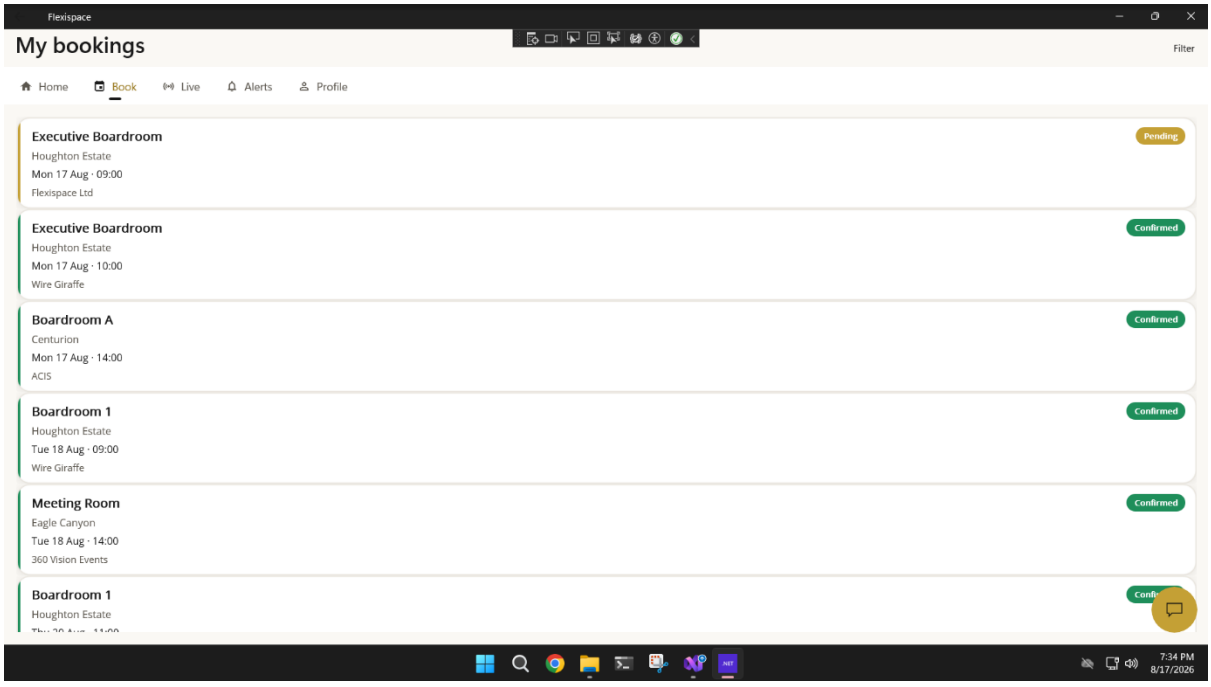




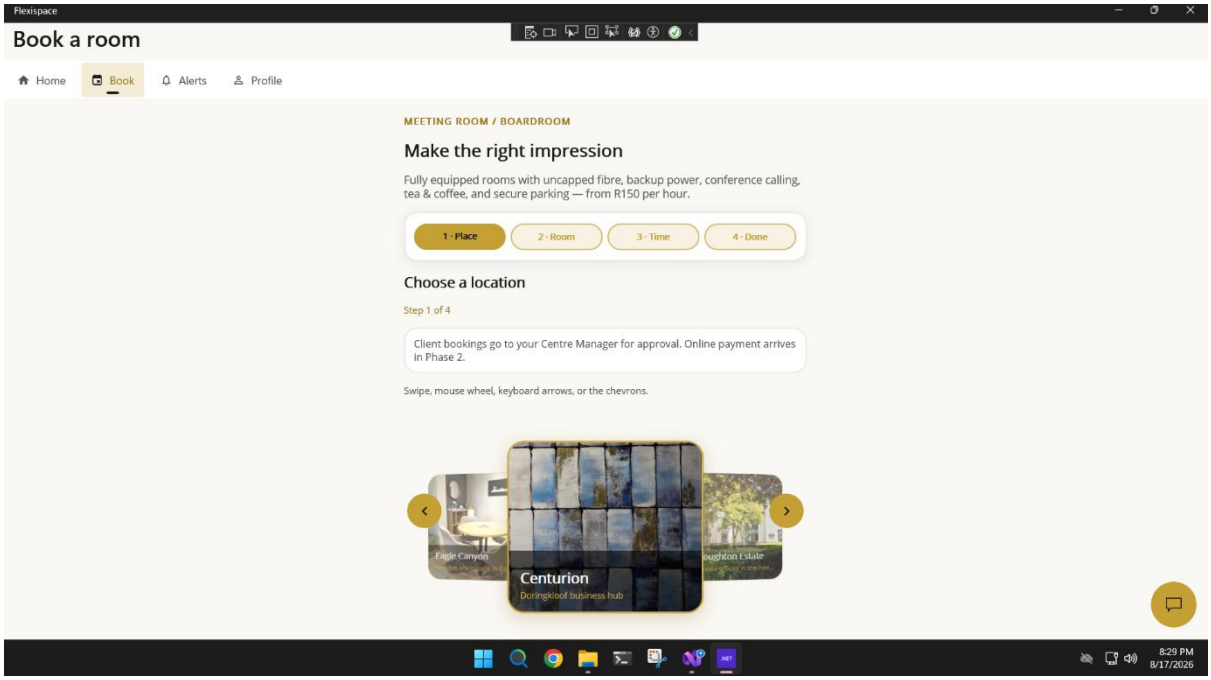
3. Alerts (All users)



4. Booking History



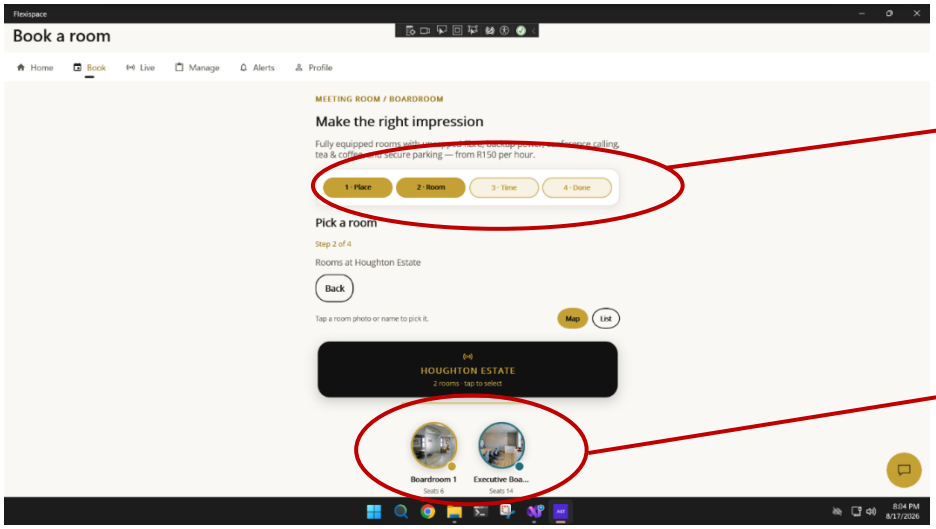
5. Choose a location



Staff

#	User Story
6	As Staff, I want to book a boardroom through a four-step wizard so that I can reserve meeting space efficiently.
7	As Staff, I want to view live room availability so that I can find an open slot before booking.
8	As Staff, I want to cancel my own bookings so that I can free up rooms I no longer need.
9	As Staff, I want my booking requests to go to a Centre Manager for approval so that room usage is controlled.

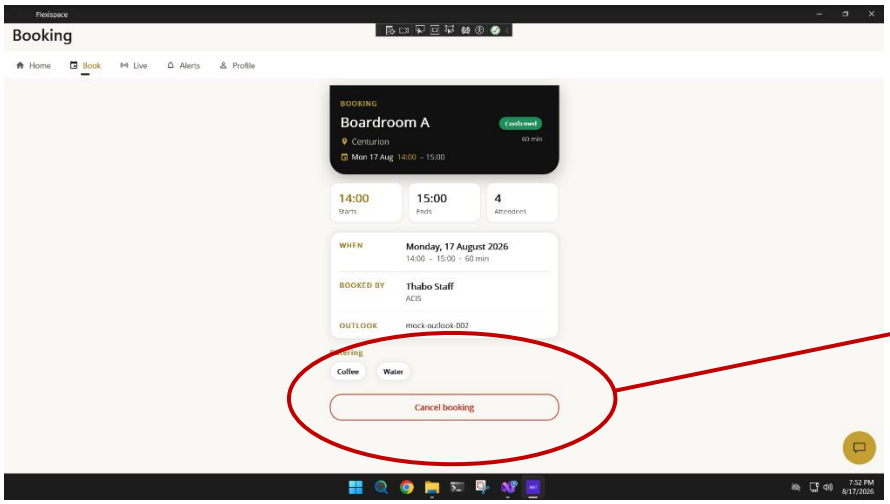
6&7. Book a boardroom



Four step wizard

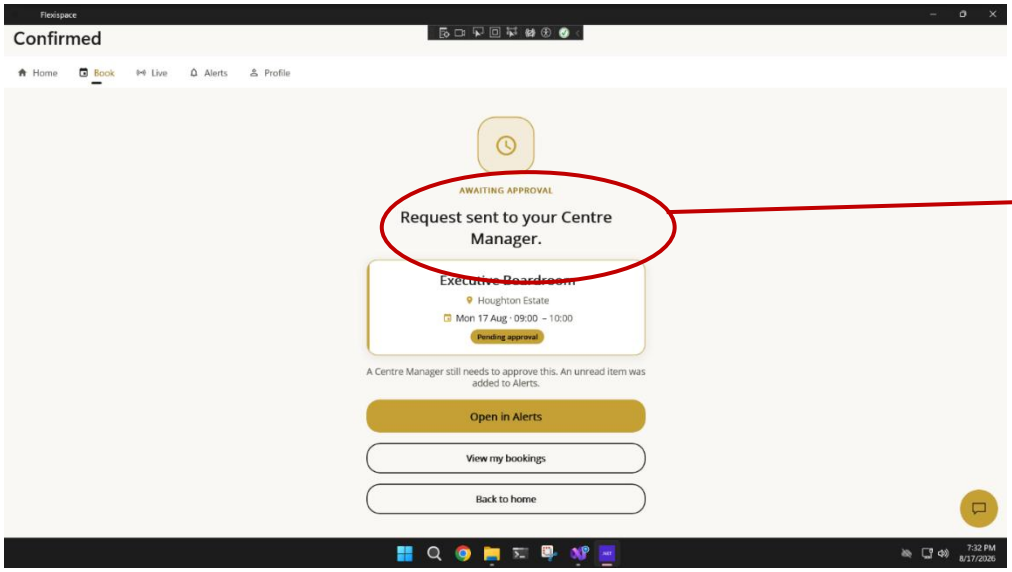
Live room availability

8. Cancel booking view



Cancel Booking

9. Booking confirmation indicating it was sent for approval for the centre manager

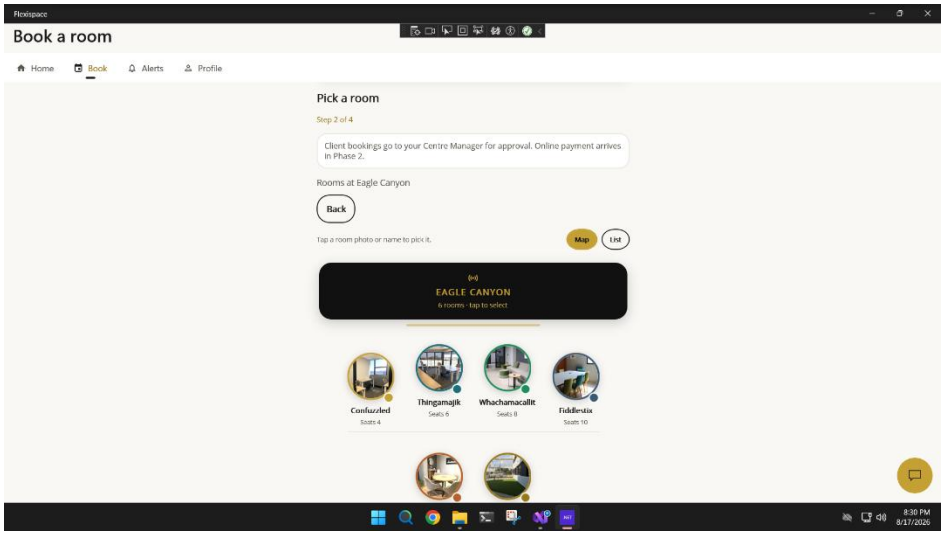


Notification stating the booking has been sent to the centre manager for approval

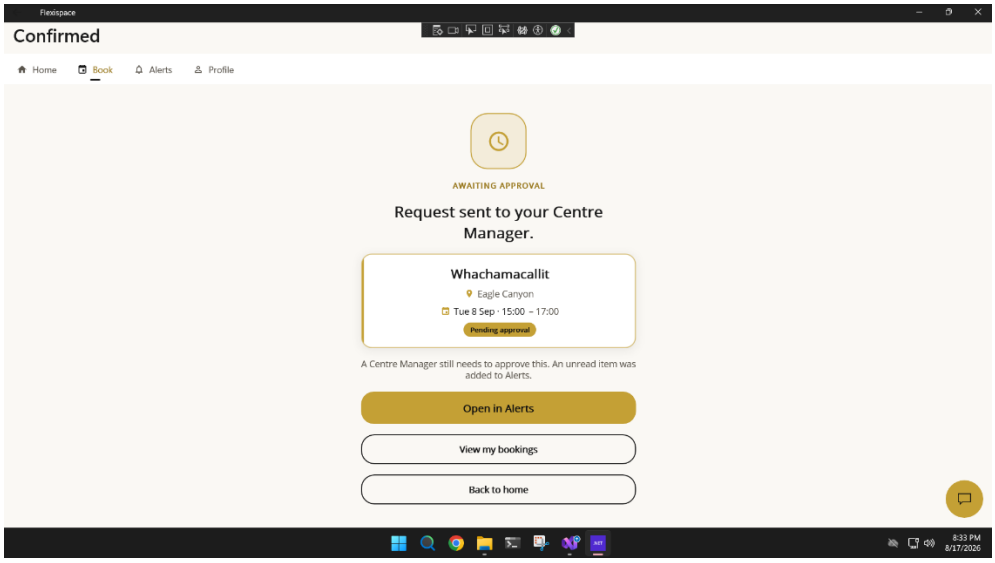
Client

#	User Story
10	As a Client, I want to self-service book boardrooms so that I can reserve space without staff assistance.
11	As a Client, I want to submit bookings for Centre Manager approval so that my reservation is validated.
12	As a Client, I want to pay online for my booking (Phase 2) so that I can complete self-service billing.
13	As a Client, I want booking confirmations in my alerts so that I have a record of my reservation.

10. Booking boardrooms

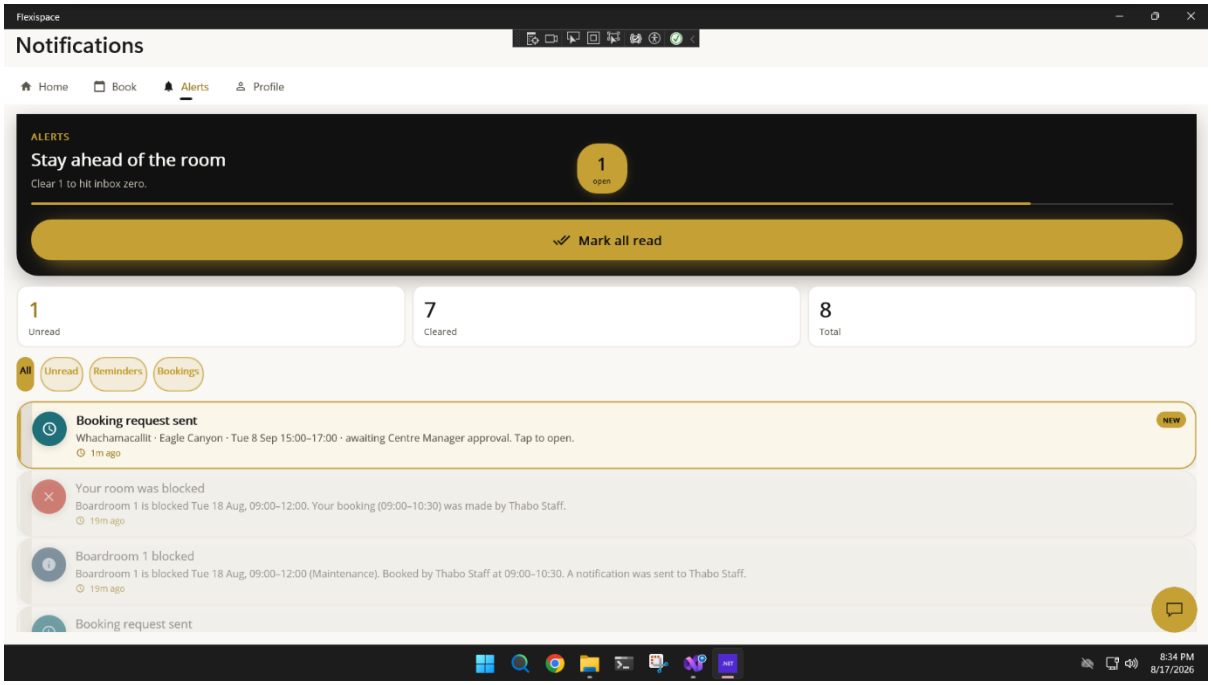


11. Booking confirmation



12. Will be integrated in Phase

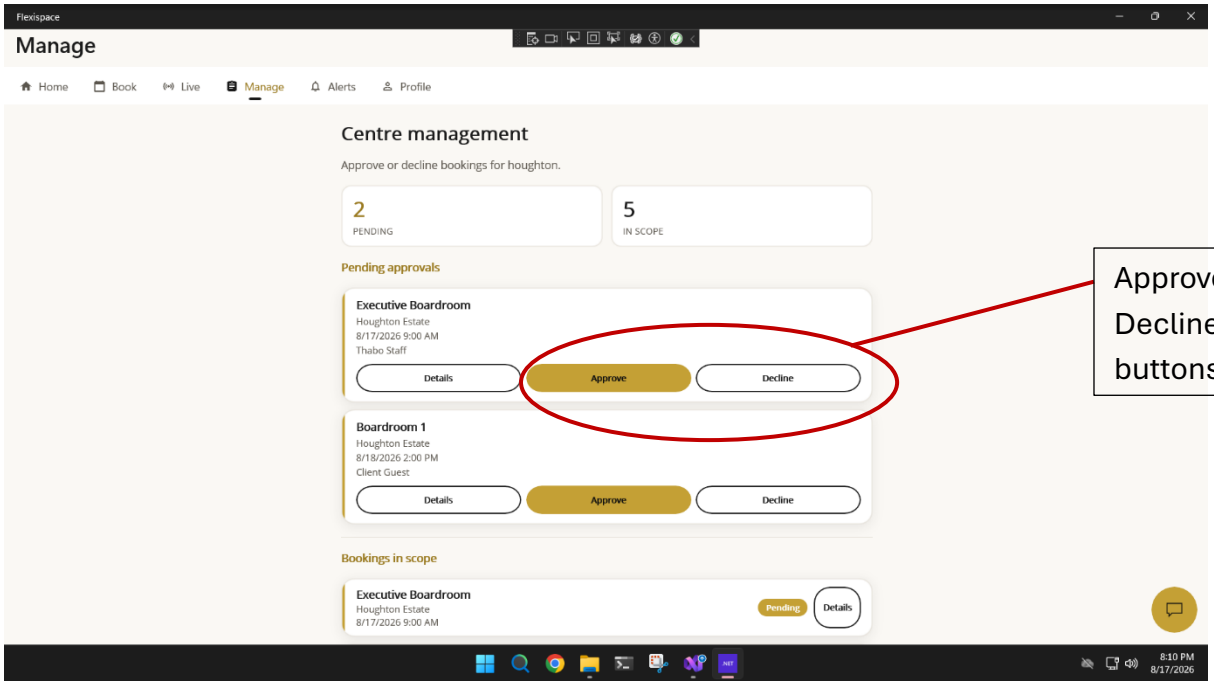
13. Booking reservation in alerts



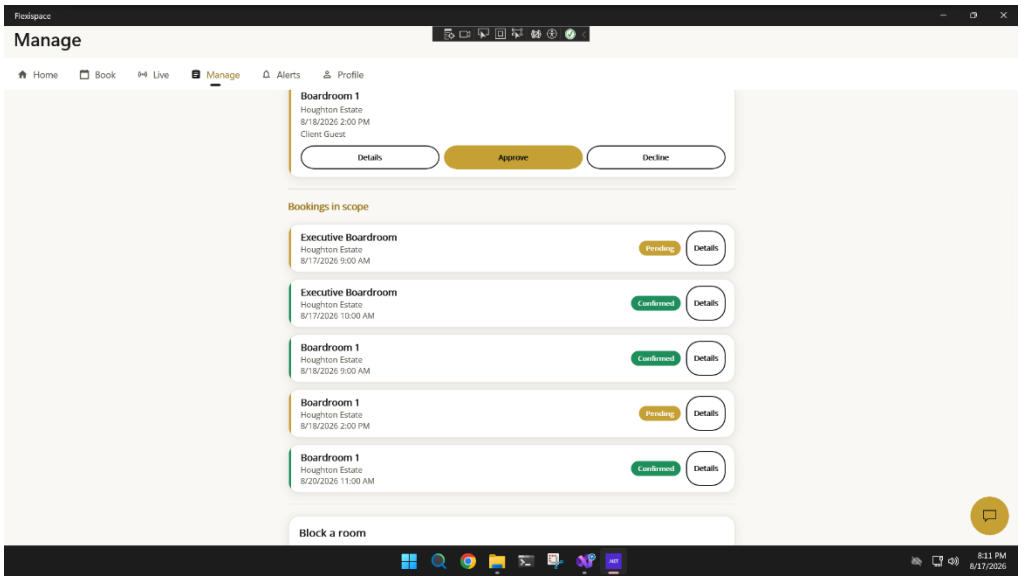
Centre Manager

#	User Story
14	As a Centre Manager, I want to approve or decline pending booking requests for my centre so that I control room allocation.
15	As a Centre Manager, I want to view and manage all bookings at my assigned location so that I have oversight of room usage.
16	As a Centre Manager, I want to edit booking details so that I can adjust reservations when needed.
17	As a Centre Manager, I want to block rooms for maintenance or events so that they cannot be booked during blocked periods.
18	As a Centre Manager, I want to book rooms only at my assigned centre so that I manage my own site.

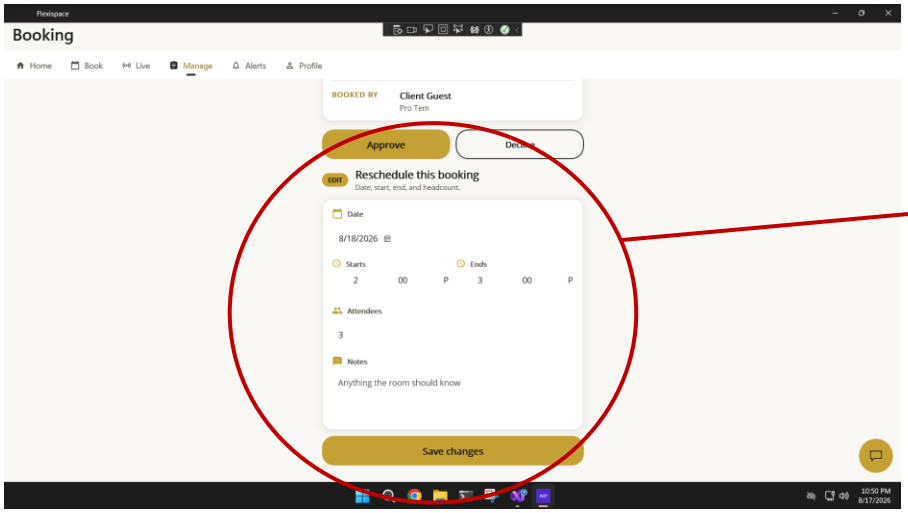
14.Pending Requests



15.All bookings at assigned location

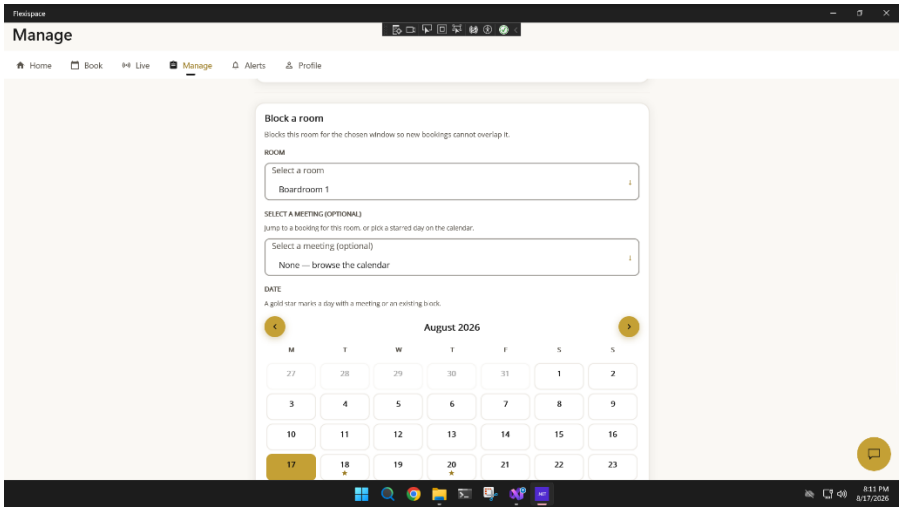


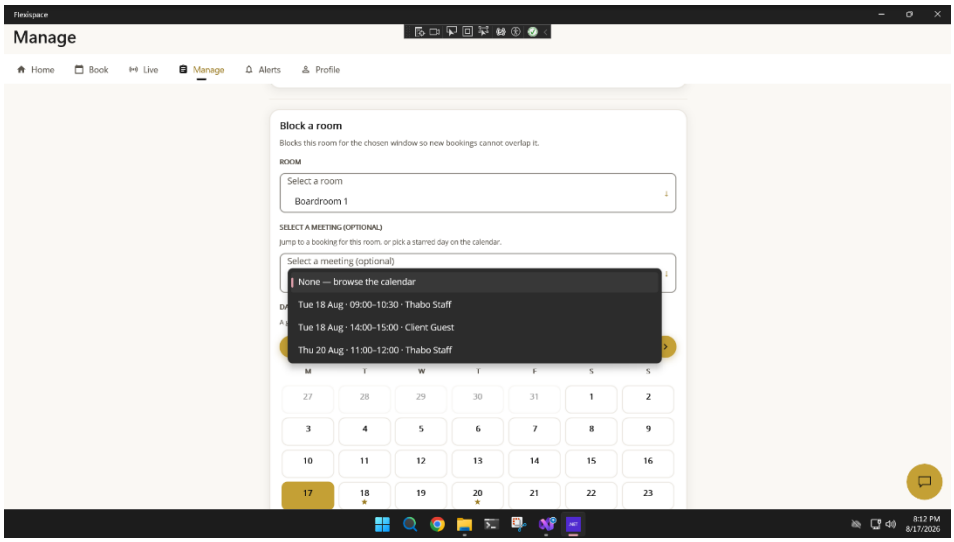
16.Edit booking details

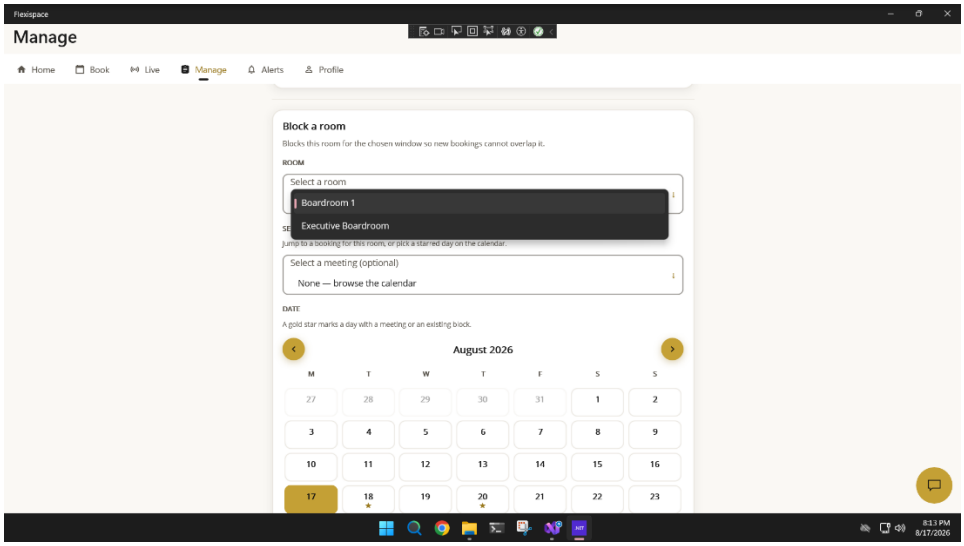


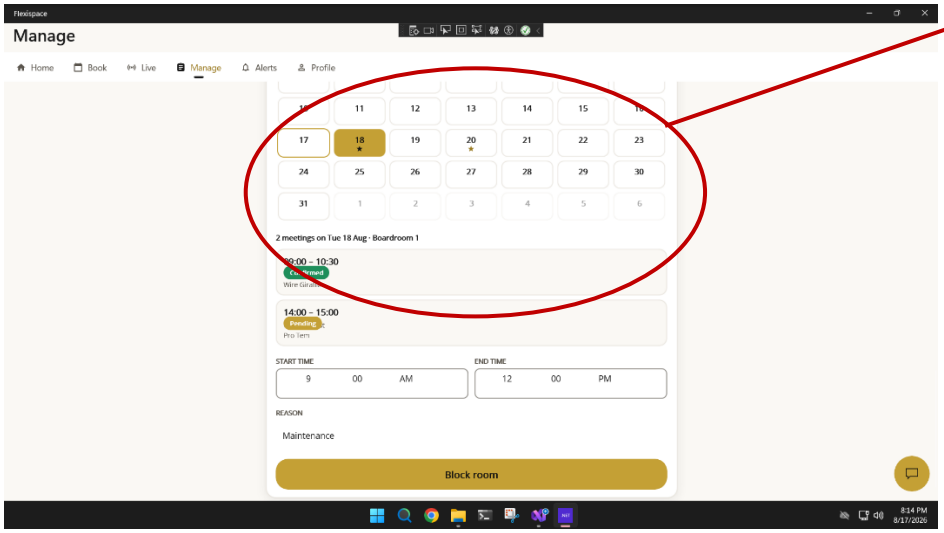
Update details section of existing bookings

17. Block a room views

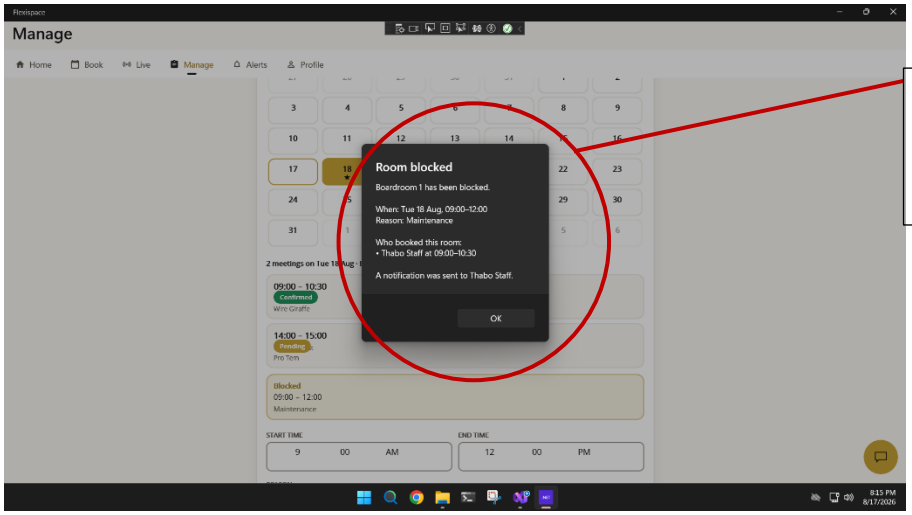






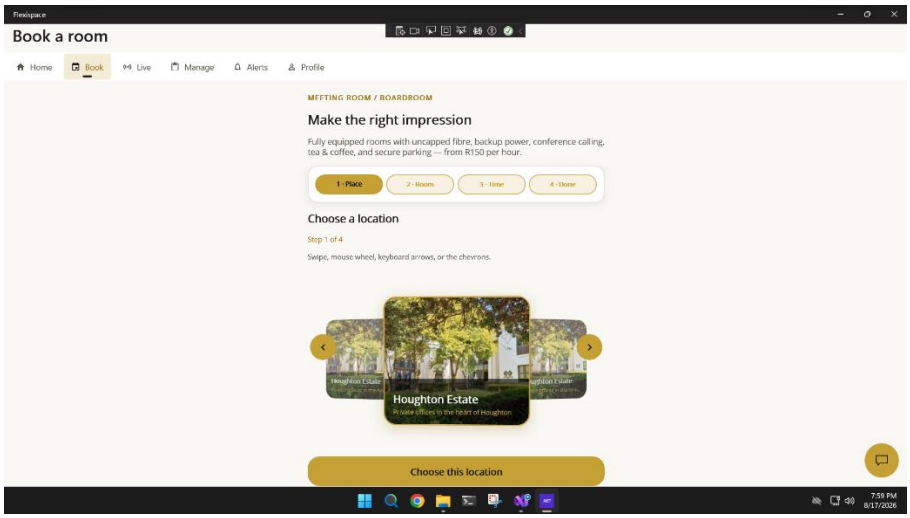


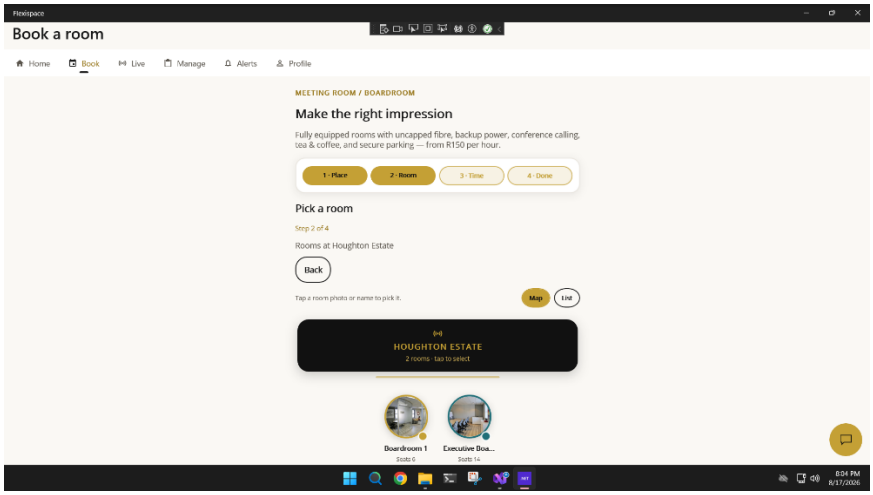
The calendar to select the date and also the stars on each date indicate that those dates have reservations



Confirmation message

18. Booking rooms view of own site only



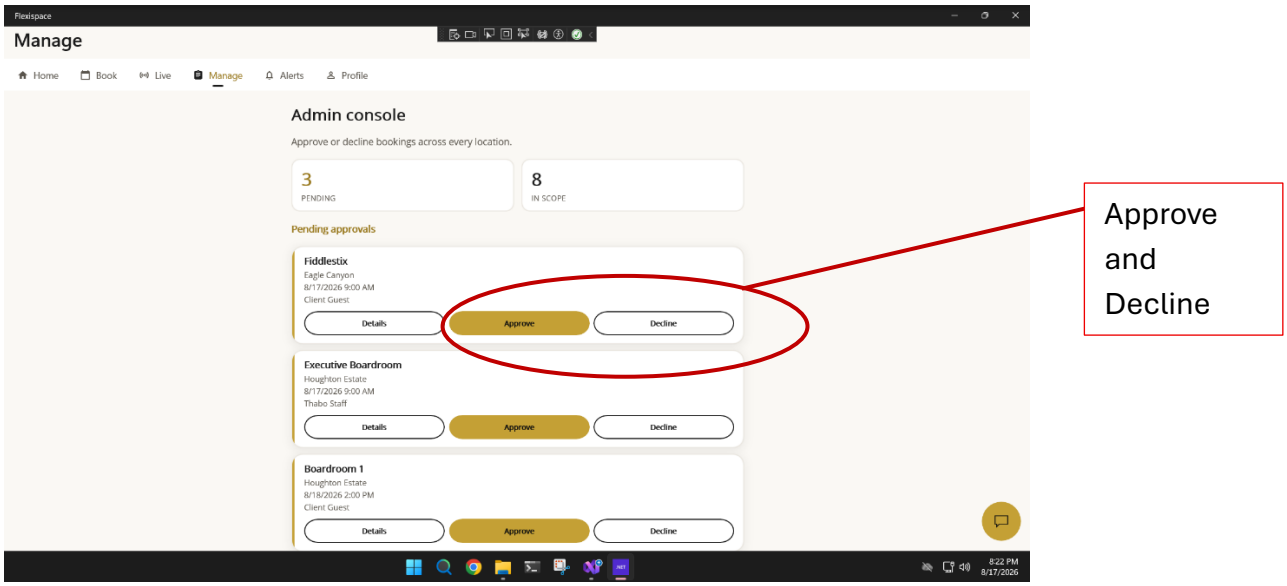


Administrator

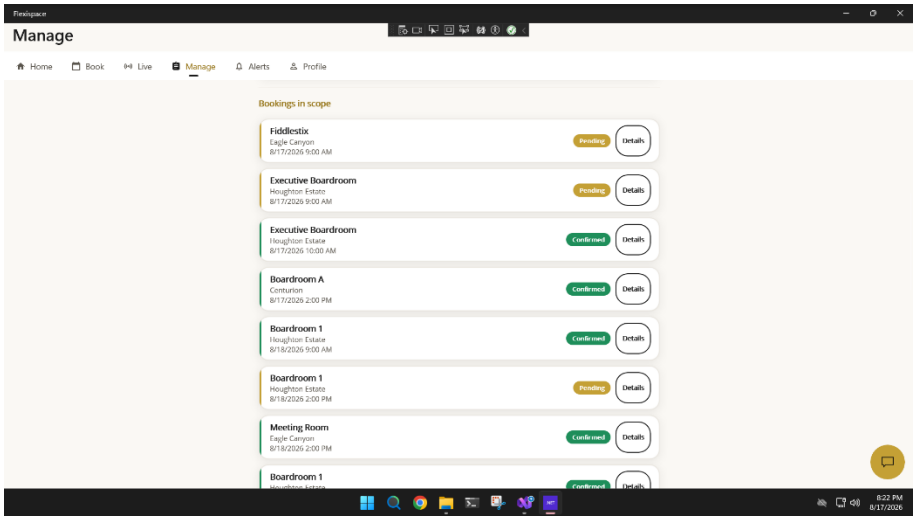
#	User Story
19	As an Administrator, I want to approve or decline bookings across all locations so that I can manage the entire portfolio.
20	As an Administrator, I want to add and remove rooms so that the room inventory stays current.
21	As an Administrator, I want to manage users so that access is controlled across the system.
22	As an Administrator, I want to view usage reports so that I can analyze utilization.

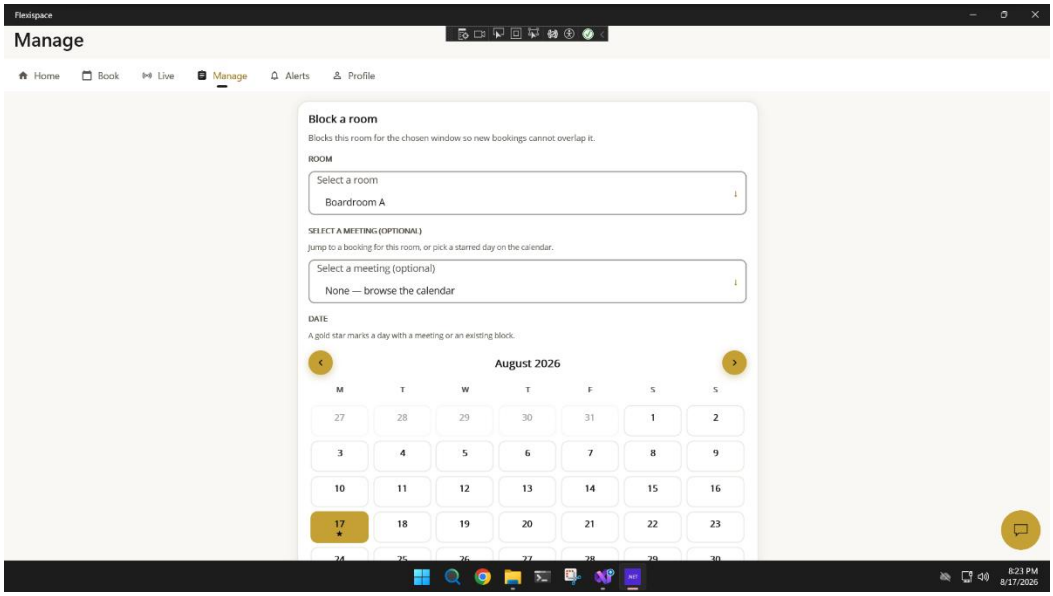
#	User Story
23	As an Administrator, I want to configure locations so that site metadata is maintained.

19. Approve or decline bookings across all locations



23. Location site metadata

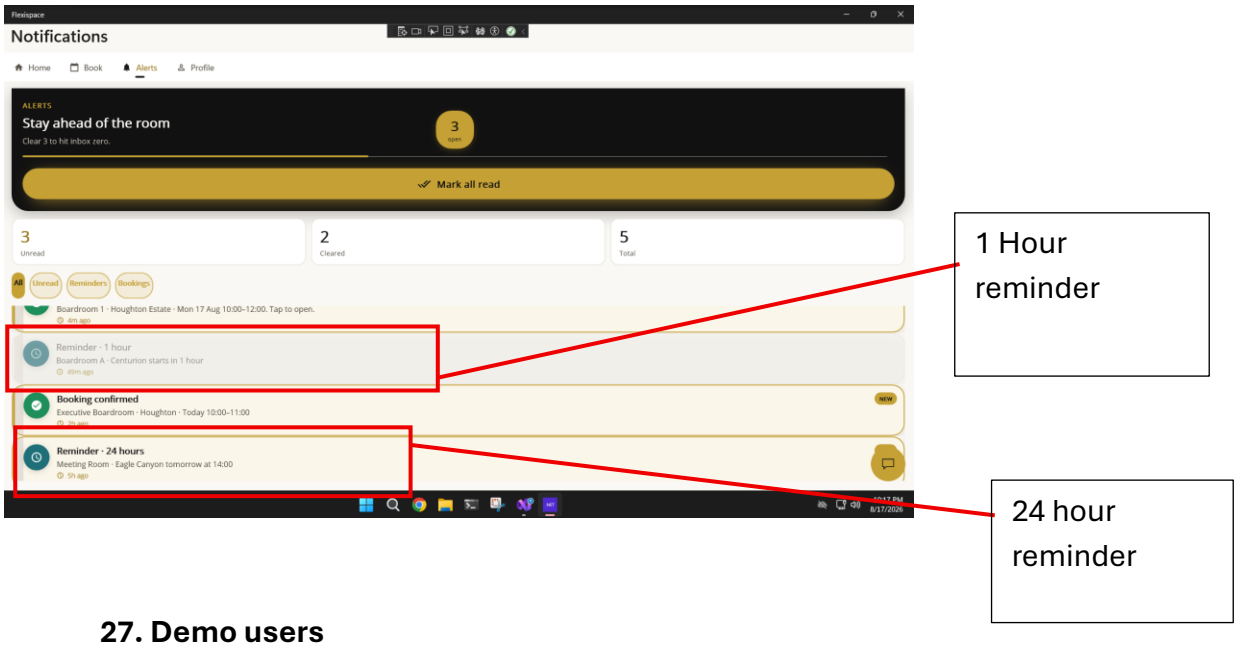




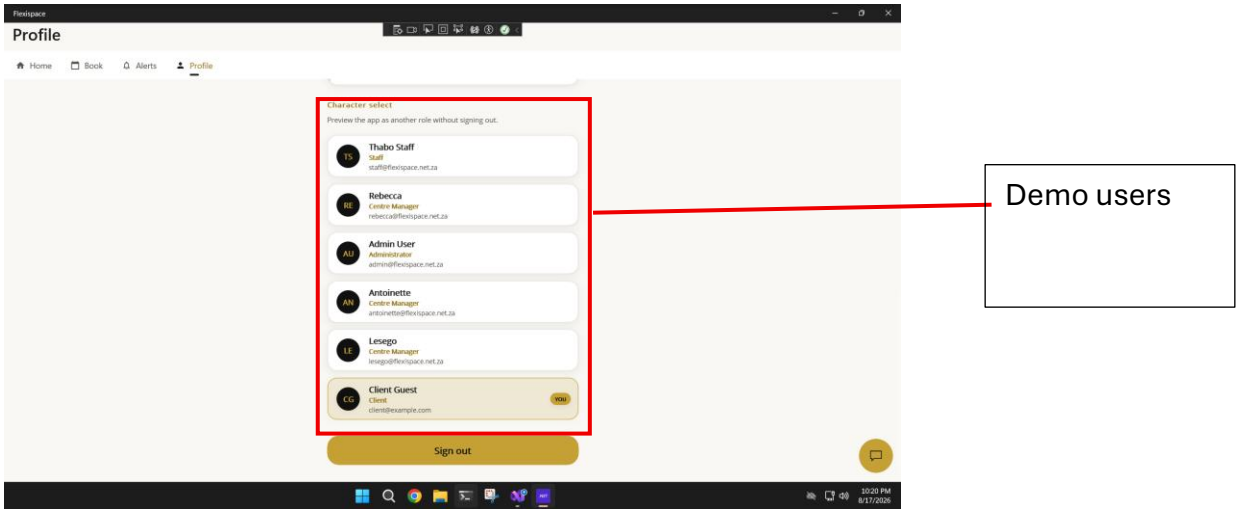
Cross-cutting / System

#	User Story
24	As a booker, I want conflict detection with suggested alternative time slots so that I can find availability when my first choice is taken.
25	As a booker, I want bookings synced to Outlook via Microsoft Graph so that meetings appear in my calendar.
26	As a user, I want reminder notifications 24 hours and 1 hour before my booking so that I don't miss my meeting.
27	As a demo user, I want to switch between roles from the profile page so that I can preview different permission levels.

26. Notifications 24 hours and 1 hour



27. Demo users



9.1.7.1 Work Breakdown Structure

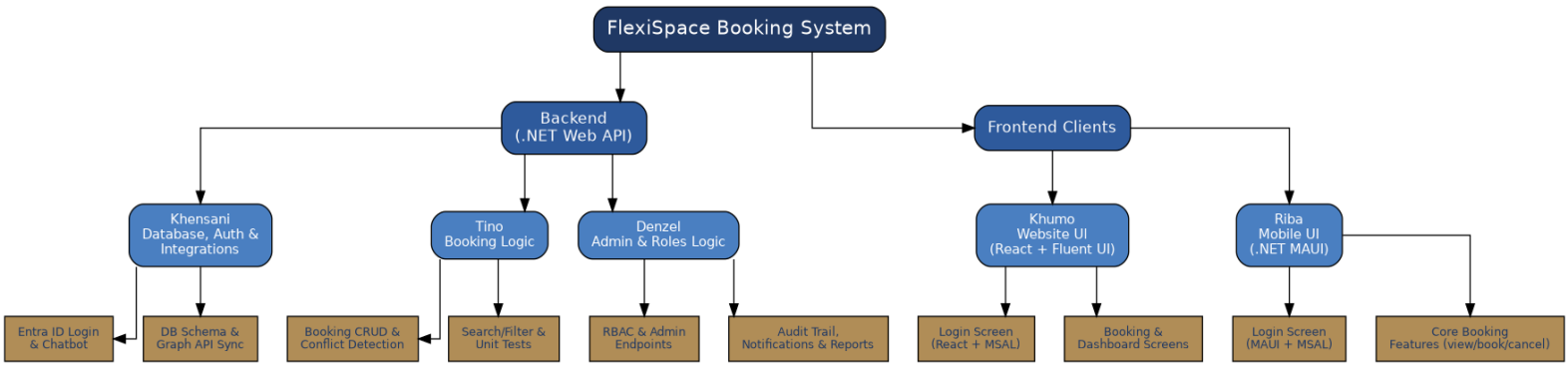


Figure 1: Work Breakdown Structure

Backend (.NET Web API)

- Khensani — Database, Authentication, Calendar Integration, Chatbot
  - Entra ID login & chatbot; database schema & Microsoft Graph API sync
- Tino — Booking Logic
  - Booking CRUD & conflict detection; search/filter & unit tests
- Denzel — Admin & Roles Logic
  - RBAC & admin endpoints; audit trail, notifications & reporting

Frontend Clients

- Khumo — Website UI (React + Fluent UI)
  - Login screen (React + MSAL); booking & dashboard screens
- Riba — Mobile UI (.NET MAUI)
  - Login screen (MAUI + MSAL); core booking features (view/book/cancel)

Cross-team Dependencies

- Tino and Denzel depend on Khensani's database schema before writing real booking/admin logic.
- Khumo and Riba depend on the API contract (endpoint list) from Khensani, Tino, and Denzel, and build against mock data until real endpoints land progressively through Phase 2.
- Website and Mobile UI need Khensani's login endpoint working before real sign-in can be tested, and need Denzel's role-check logic before admin-only screens can be safely restricted.

9.1.8 Non-functional Requirements

A. Security

Req ID	What it does	Priority
NFR-SEC-001	Server-side role checks - don't trust the front-end	High
NFR-SEC-002	Use parameterized queries to block SQL injection attacks	High
NFR-SEC-003	Store ALL secrets in environment variables - never in code	High
NFR-SEC-004	Show generic error messages to users (not the ugly stack traces)	High
NFR-SEC-005	Never log passwords, tokens, or personal info in plain text	High
NFR-SEC-006	Rate limiting and lockouts on login attempts - prevent brute force	High
NFR-SEC-007	Mobile app stores tokens securely (Keychain/Keystore)	High
NFR-SEC-008	HTTPS only - no unencrypted communication	High
NFR-SEC-009	No API keys hardcoded in the mobile app	High
NFR-SEC-010	POPIA compliance - document what personal data we store and why	High
NFR-SEC-011	CORS configured so only our website can call the API	High

B. Performance

Req ID	What it does	Priority
NFR-PER-001	Check availability and conflicts in under 3 seconds	High
NFR-PER-002	Load booking lists (with pagination) in under 2 seconds	High

NFR-PER-003	Support 25+ users at once without slowing down	Medium
NFR-PER-004	Handle 10,000+ historical bookings without performance issues	Medium

C. Reliability

Req ID	What it does	Priority
NFR-REL-001	Single source of truth - no chance of double bookings	High
NFR-REL-002	99.5% uptime during business hours (Mon-Fri, 8am-6pm)	High
NFR-REL-003	Outlook sync fails over gracefully (webhooks -> polling)	Medium
NFR-REL-004	Show friendly error messages when network fails	High

D. Usability

Req ID	What it does	Priority
NFR-USAB-001	Book a room in under a minute	High
NFR-USAB-002	Website uses Fluent UI so it looks and feels like Microsoft	High
NFR-USAB-003	Mobile app works on both iPhone and Android	High
NFR-USAB-004	Clear error messages that actually tell you what went wrong	High
NFR-USAB-005	WCAG 2.1 AA accessibility guidelines	Medium

E. Scalability

Req ID	What it does	Priority
NFR-SCA-001	Easy to add new FlexiSpace locations later	High
NFR-SCA-002	Database design allows adding new rooms without breaking stuff	High
NFR-SCA-003	Hosted on Azure so we can scale up when needed	High

9.1.9 Analysis Artifacts

This section breaks down what we learned from FlexiSpace's requirements and how we turned that into a solid plan for building the booking system.

9.1.9.1 Domain Modelling

Domain modelling maps out all the important “things” in the system and how they relate to one another. Below is the refined domain class diagram identifying the key concepts.

Domain Class Diagram

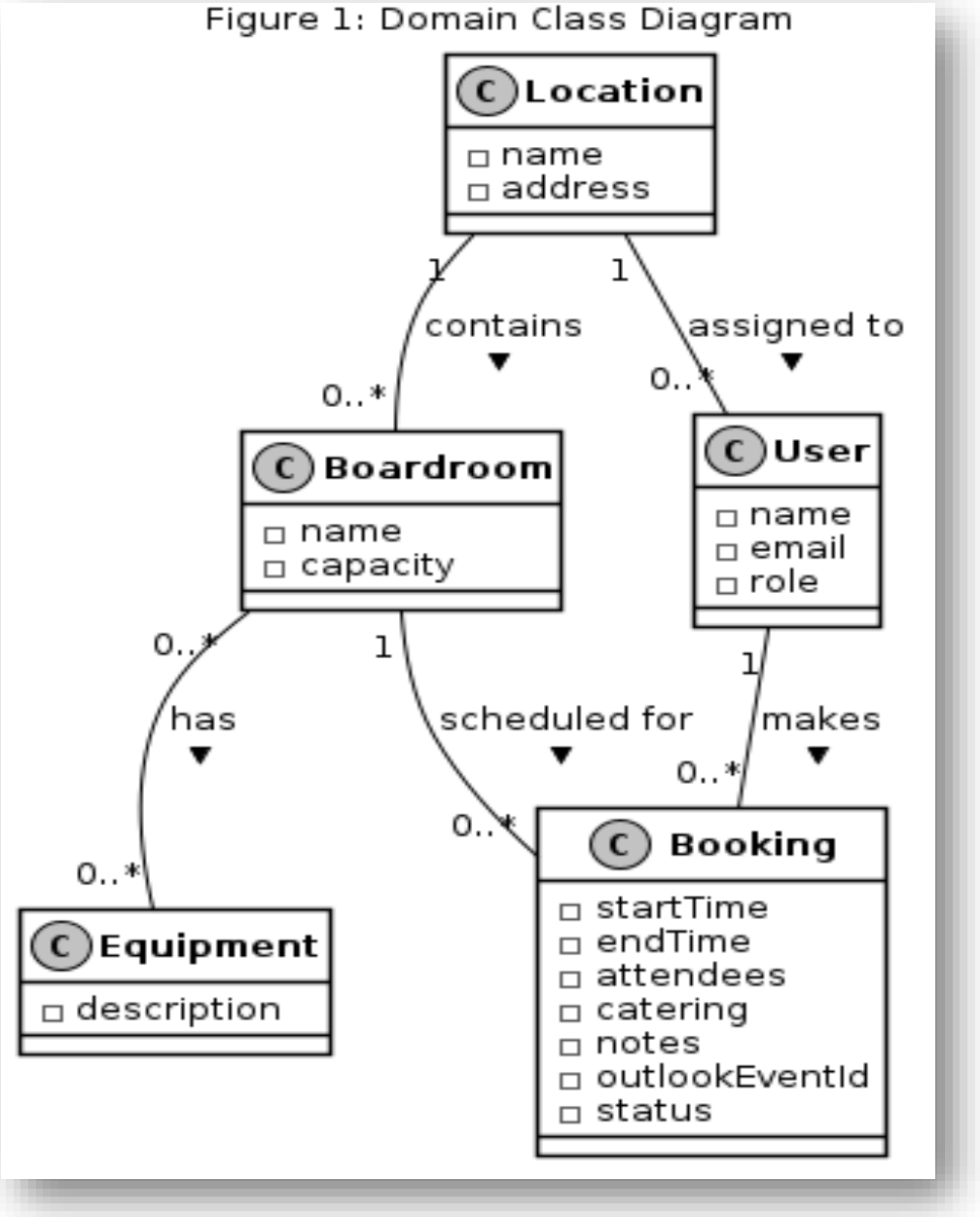


Figure 2: Domain Class Diagram

Core Domain Classes Explained

- **Location:** Represents each FlexiSpace site, such as Centurion, Houghton, or Eagle Canyon. Each location has a name and address, allowing new locations to be added later without breaking the system.
- **Boardroom:** The actual meeting rooms people want to book. Each room has a name, capacity limit, and belongs to a specific location. Rooms can also have equipment assigned to them, helping users find exactly what they need.

- **Equipment:** Covers items found in a boardroom, such as TVs, projectors, whiteboards, video conferencing gear, and HDMI cables. Users can filter rooms based on what equipment they need.
- **User:** Anyone using the system. There are different user types: Administrators, Centre Managers, regular Staff, and Clients. Each user has a role that controls what they can and cannot do.
- **Booking:** The heart of the system. A booking records who booked what room, when, for how many people, any catering needed, and special notes. Each booking has a status (Pending, Confirmed, Cancelled, or Blocked) and connects to an Outlook event so both systems stay in sync.

Key Relationships

- **Locations and Boardrooms:** One location can have many boardrooms, but each boardroom only belongs to one location.
- **Boardrooms and Bookings:** One room can have many bookings over time, but each booking is for just one room.
- **Users and Bookings:** One user can make many bookings, and each booking is tied to the person who made it.
- **Boardrooms and Equipment:** Rooms can have multiple equipment items, and the same equipment can be in multiple rooms.

9.1.9.2 Design Artifacts

Moving from the concept map to implementation detail — what the code will actually look like.

Design Class Diagram

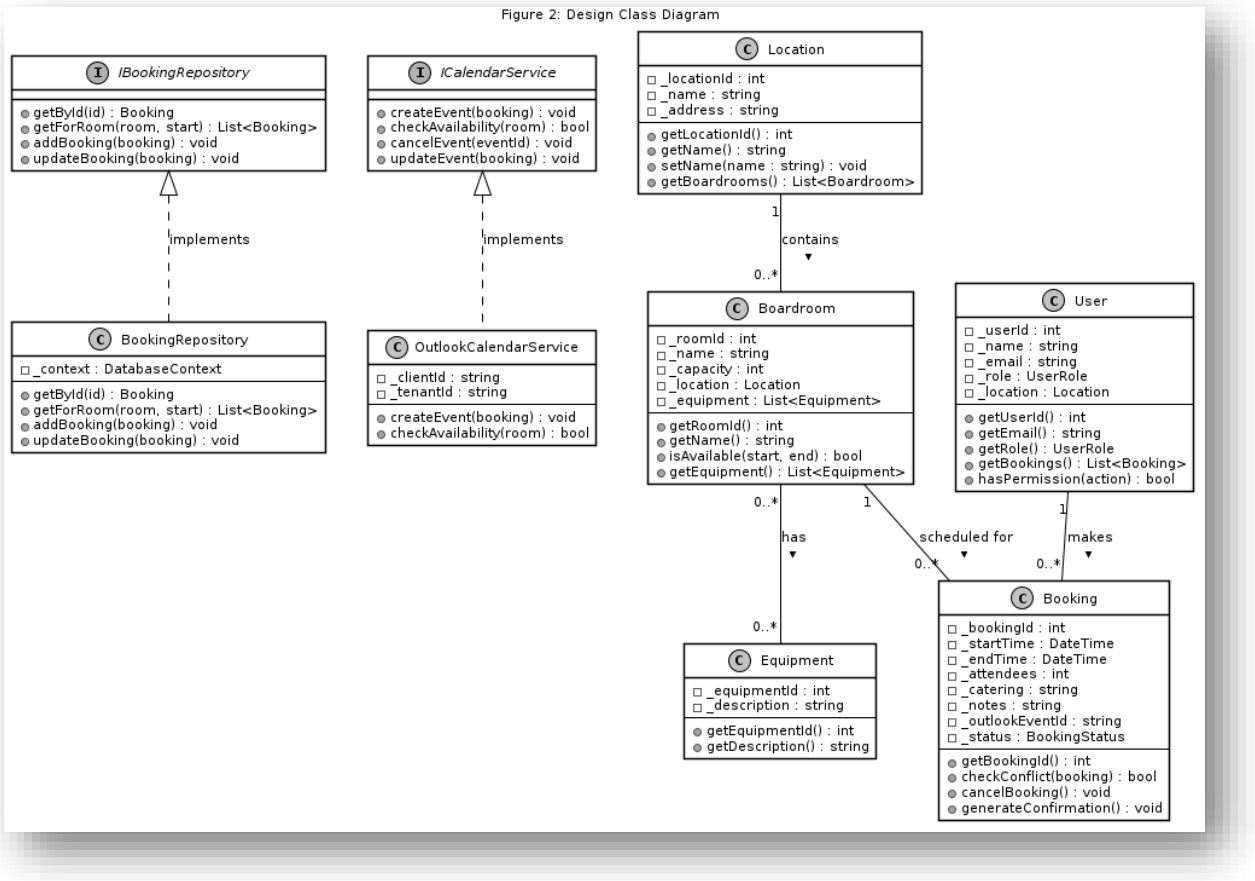


Figure 3: Design Class Diagram

Implementation Classes

- **Location Class:** Stores the location ID, name, and address privately. Public methods let other parts of the system retrieve this information, including a method to fetch all boardrooms at that location.
- **Boardroom Class:** Holds the room ID, name, capacity, its location, and a list of equipment. Key methods include checking room availability at a specific time and getting the equipment list.
- **Equipment Class:** A simple class with an ID and description, referenced by other classes when listing what is in a room.
- **User Class:** Stores user ID, name, email, role, assigned location, and bookings. Important methods include checking permissions and retrieving booking history.
- **Booking Class:** Tracks everything about a reservation, including times, attendee count, catering, notes, Outlook event ID, and status. Key methods check for booking conflicts, cancel the booking, and generate confirmation emails.

- **OutlookCalendarService:** Handles all communication with Microsoft Outlook through the Graph API, including creating events, checking availability, and cancelling events.
- **BookingRepository:** Manages all database operations for bookings, allowing other parts of the system to retrieve and save booking data without dealing with database details directly.

Keeping Things Clean

- **Dependency Injection:** We use so our controllers don't have to create the services themselves. For example, BoardroomController uses IBoardroomService, and the actual BoardroomService is given to it by .NET. This keeps the code less dependent on specific classes and also makes it easier to test
- **Calendar Service Abstraction:** An interface is used for Outlook integration. The booking logic does not need to know whether it is talking to Outlook, Google Calendar, or another provider — it simply uses the interface.
- **Service Layer:** The BookingService coordinates everything. It checks for conflicts, talks to the repository and calendar service, and ensures business rules are followed before saving anything.

9.1.10 Implementation

This section covers how the system will be built and deployed.

9.1.10.1 Sequence Diagrams

Sequence diagrams show how different parts of the system interact during specific scenarios.

User Books a Boardroom (Success)

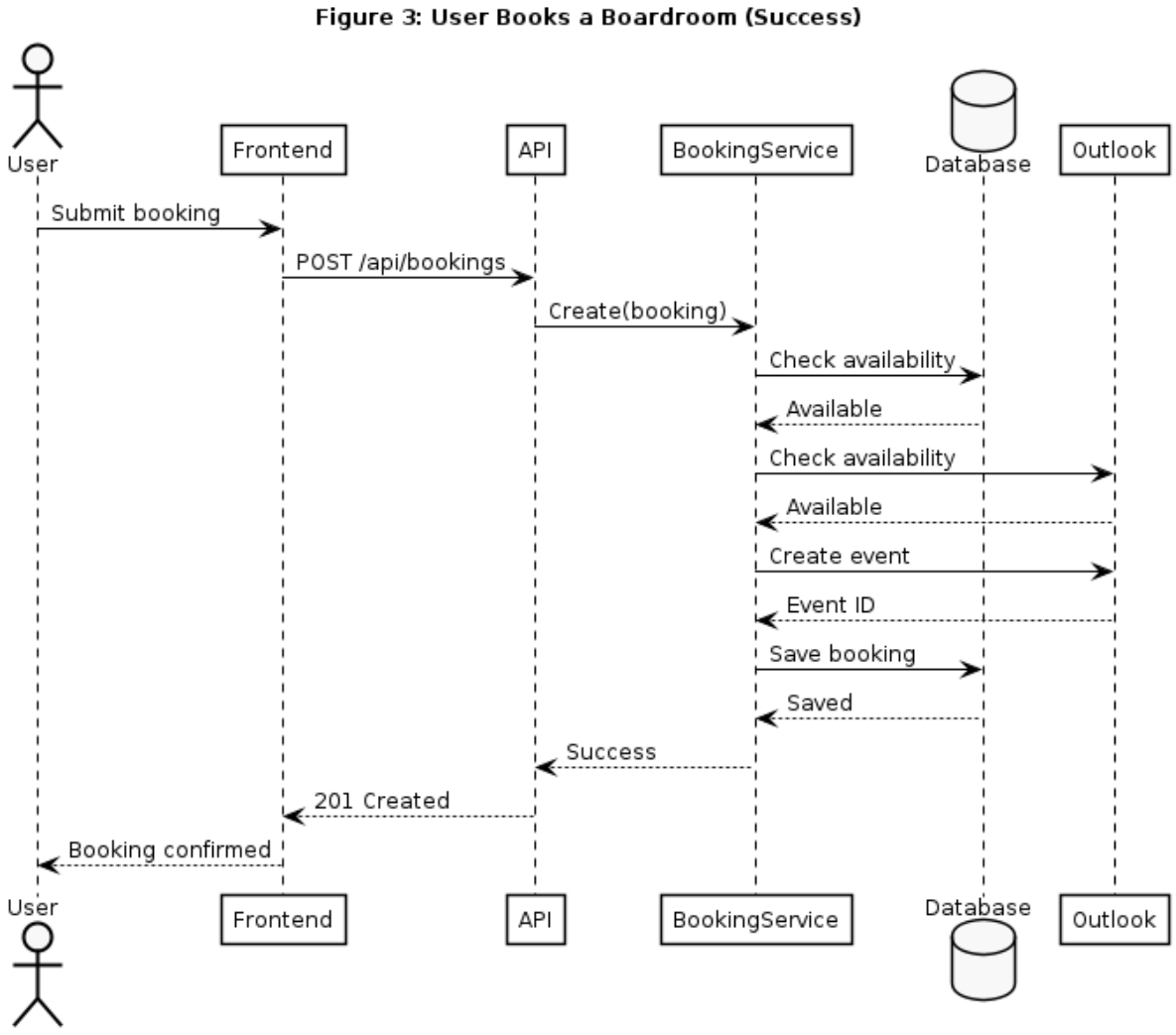


Figure 4: User Books a Boardroom (Success)

- The user fills in the booking form and submits it.
- The frontend sends the request to the API.
- The BookingService checks if the room is available in the database.
- It also checks availability in Outlook.

- If everything is clear, it creates an event in Outlook.
- It saves the booking in the database.
- The user receives confirmation.

Booking Conflict Detection

Figure 4: Booking Conflict Detection

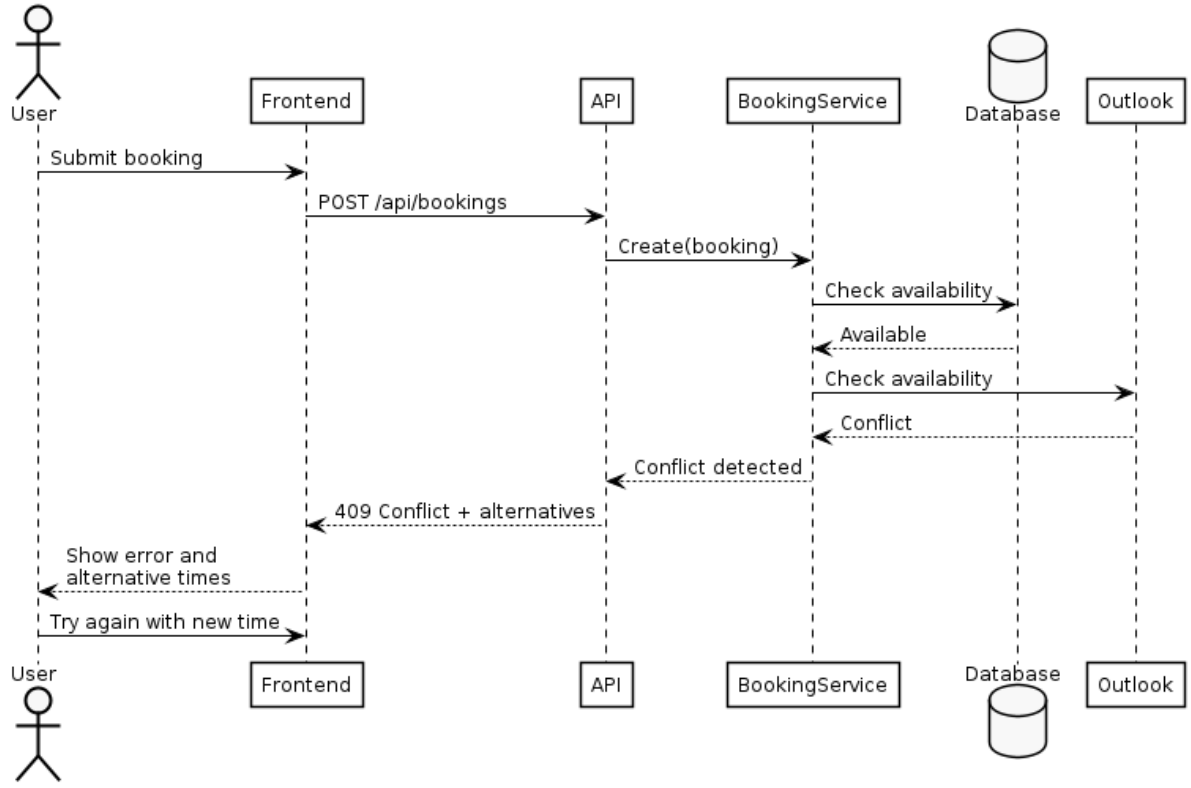


Figure 5: Booking Conflict Detection

- The user submits a booking.
- The database shows the room is available.
- Outlook reports that the room is already occupied.
- The system returns a conflict error with alternative time suggestions.
- The user picks a new time and tries again.

9.1.10.2 State Diagram

State diagrams show how an object changes state over time.

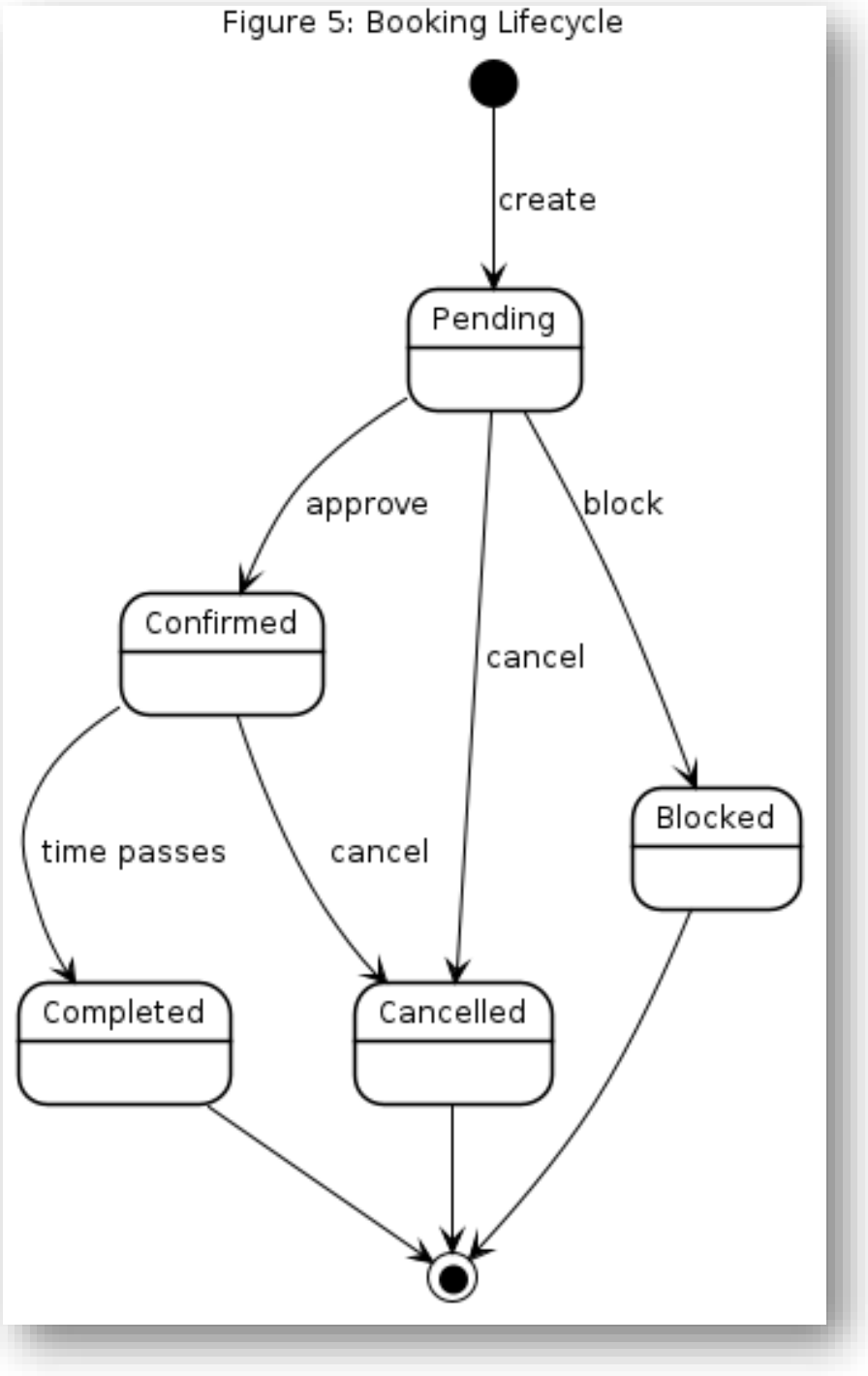


Figure 6: Booking Lifecycle

- **Pending:** Freshly created but not yet confirmed. Can be approved, cancelled, or blocked.

- **Confirmed:** Approved and official. The room is locked in.
- **Cancelled:** The booking is dead. The room opens up again.
- **Blocked:** Admin has locked it for maintenance or special events.
- **Completed:** The time has passed. Archived for records but can no longer be changed.

9.1.10.3Deployment Strategy

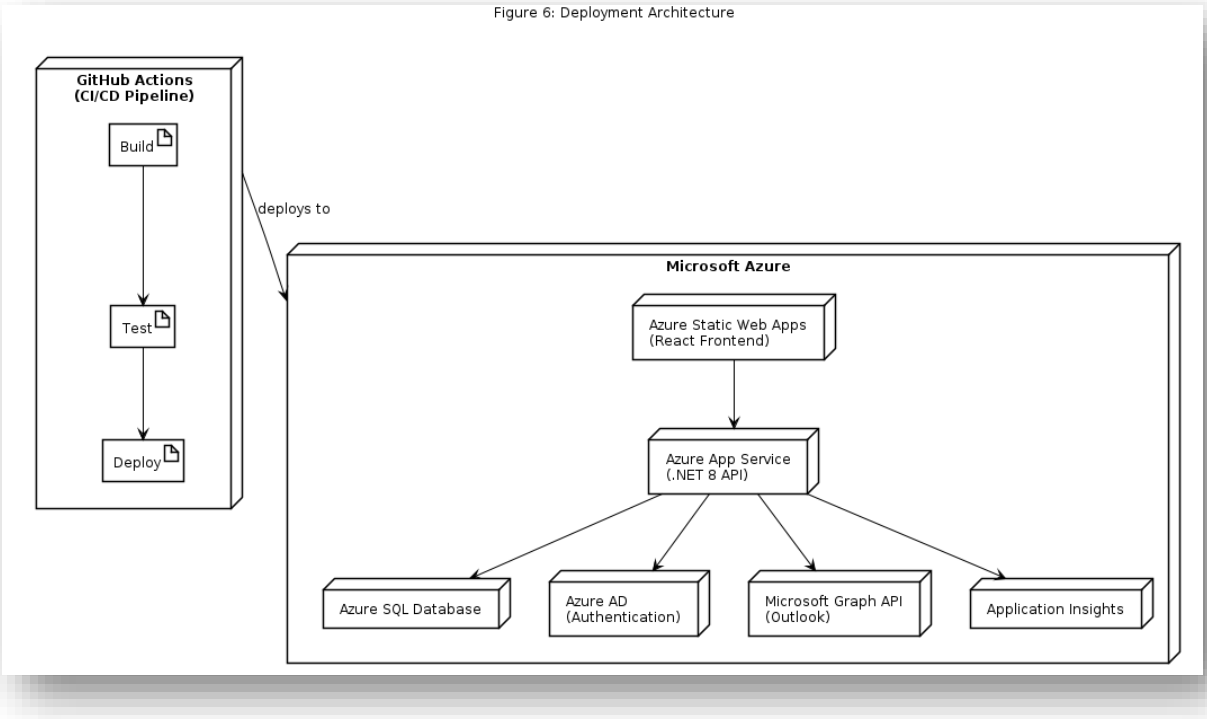


Figure 7: Deployment Architecture

Deployment Plan

Where It Will Live: Everything is hosted on Microsoft Azure. The backend (.NET 8 API) runs on Azure App Service, the frontend (React) on Azure Static Web Apps, and the database on Azure SQL. Azure Active Directory handles logins.

How It Will Be Deployed: The whole deployment is automated through GitHub Actions. When code is pushed, it automatically builds, tests, and deploys. If something breaks, the previous version can be restored instantly.

Rollout Plan

Phase	Description
Phase 1 – Pilot	Start with just the Centurion location and a small group of staff to catch issues early.
Phase 2 – Full Launch	After fixes, deploy to all locations during off-hours.
Phase 3 – Support	Dedicated support team monitoring for problems.

Technical Stack

- Frontend: React with Microsoft Fluent UI
- Backend: .NET 8 Web API
- Database: Microsoft SQL Server
- Integration: Microsoft Graph API for Outlook
- Hosting: Microsoft Azure
- CI/CD: GitHub Actions

9.1.11 Data Schema

9.1.11.1 Technology Choice:

For the FlexiSpace Booking System, we chose **Microsoft Azure SQL Server** as our database technology. FlexiSpace uses **Azure SQL Database** as its main data store, with **Entity Framework Core 8** used as the ORM and **EF Core Code-First Migrations** used to manage changes to the database structure.

- **Azure SQL Database** – We chose this because FlexiSpace has a lot of related data. For example, a **Location can have multiple Boardrooms**, and a **Boardroom can have multiple Bookings.** A relational database makes it easier to manage these relationships and maintain data integrity. Using Azure SQL also fits with our Azure-based environment and means we do not have to manage the database server ourselves.
- SQL Server is also useful for one of the most important requirements of FlexiSpace: **preventing double bookings**. Before a booking is saved, the system needs to check the selected boardroom, date and time against existing bookings and blocked periods. The relational structure makes these types of queries easier to manage.
- **Entity Framework Core** – We use EF Core to connect our C# entity classes to the database tables. This fits with our **ASP.NET Core Web API** and Clean Architecture structure, with the database and DbContext managed within
- **EF Core Migrations** – Migrations allow us to keep track of changes made to the database structure. This means changes can be stored in source control and the database can be recreated consistently when needed.

Alternative considered: We considered **Azure Cosmos DB**, which is a NoSQL database, because of its scalability and document-based structure. However, SQL Server was a better fit for FlexiSpace because our system depends heavily on relationships between tables and needs accurate queries for things such as booking availability and conflict detection.

Scalability: So, using the relational structure it also us to expand as the system grows by adding more locations, boardrooms, users, and bookings without needing to redesign the whole database. This is important because the project is intended to support future office locations.

9.1.11.2 ERD of FlexiSpace Database

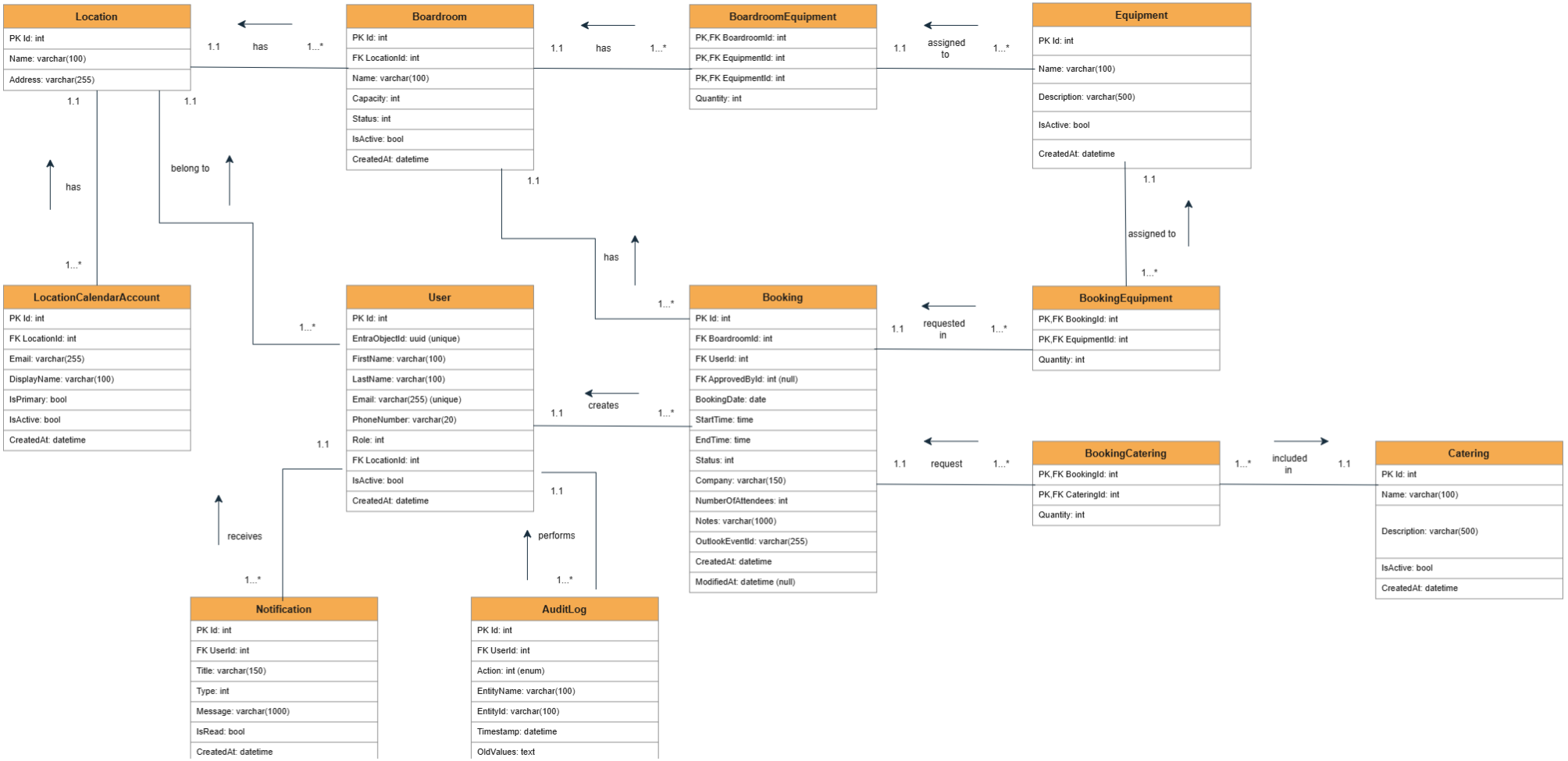


Figure 8: FlexiSpace ERD

ERD Explanation

Normalisation

Third Normal Form (3NF): every non-key attribute depends only on its table's primary key — no repeating groups, no derived data stored redundantly. E.g. a Booking doesn't duplicate Boardroom.Name or Location.Address; it stores BoardroomId and joins when needed, hence why we created junction tables (Elmasri & Navathe, 2016).

Explanation of the Relationships

Location as the root entity

Relationship	Cardinality	Explanation
Location → Boardroom	1: many	Each of the three FlexiSpace sites (Eagle Canyon, Houghton, Centurion) has multiple boardrooms. A Boardroom cannot exist without a Location (Restrict delete).
Location → LocationCalendarAccount	1: many	Supports sites with more than one monitored Outlook mailbox — e.g. Eagle Canyon has two Centre Managers, each needing their own synced calendar account.
Location → User	1: many	Every user (Centre Manager, staff, admin) is based at one FlexiSpace location

Booking as the central transaction

Relationship	Cardinality	Explanation
Boardroom → Booking	1: many	A boardroom can have many bookings over time, but each booking is for exactly one room. Deleting a Boardroom with existing bookings is blocked (Restrict)
User → Booking (booked by)	1: many	The user who made the booking. Required — every booking must have a requester.

Booking add-ons (Equipment and Catering requests)

Relationship	Cardinality	Explanation
Booking ←→ Equipment Via BookingEquipment	many: many	equipment requested for one specific booking — separate from BoardroomEquipment because a booking might request equipment the room doesn't have installed (e.g. an extra projector borrowed from another room). Deleting the Booking cascades and

		removes the request; deleting Equipment is restricted so historical booking records aren't silently corrupted.
Booking ←→ Catering Via BookingCatering	many: many	A booking can request multiple catering packages, each with its own Quantity (e.g. 10 units of "Boardroom Lunch Pack" for a Houghton meeting). Same cascade/restrict logic as above.

System-managed entities

Relationship	Cardinality	Explanation
User → Notification	1: many	Notifications (booking confirmations, reminders) are generated per user. Deleting a User cascades to their notifications — no orphaned messages.
User → AuditingLog	1: many	Every audit entry records which user performed an action. Deletion is restricted (Restrict), so a user can never be deleted while their audit trail still references them — audit history must be permanent.

9.1.12 Architecture Artifacts

9.1.12.1 Design Patterns

FlexiSpace's backend applies two complementary design patterns: the **Service Layer Pattern** and **Dependency Injection**.

Service Layer Pattern

FlexiSpace uses the Service Layer Pattern to separate the API controllers form the application's operations and database access. So, the controllers handle HTTP request and responses, while the service classes handle operations such as creatin, updating and retrieving bookings.

For example, when a booking is created, the **BookingConroller** calls the **IBokingService**, which is implemented by **BookingService**. The service then uses **ApplicationDbContext** to communicate with the database.

Dependency Injection

Services such as IBoardroomService and IBookingService are registered in the DI container when the application starts and then passed into the controllers through their constructors (Microsoft, 2026).

This means the controllers only depend on the **interface**, rather than directly depending on the actual service class. For example, BoardroomController works with IBoardroomService without needing to know how BoardroomService is implemented.

This makes the system easier to **test and maintain**, because we can replace the real service with a mock version when testing, or change the service implementation without having to change the controller.

UML Diagram Example of Service Layer and Dependency Injection

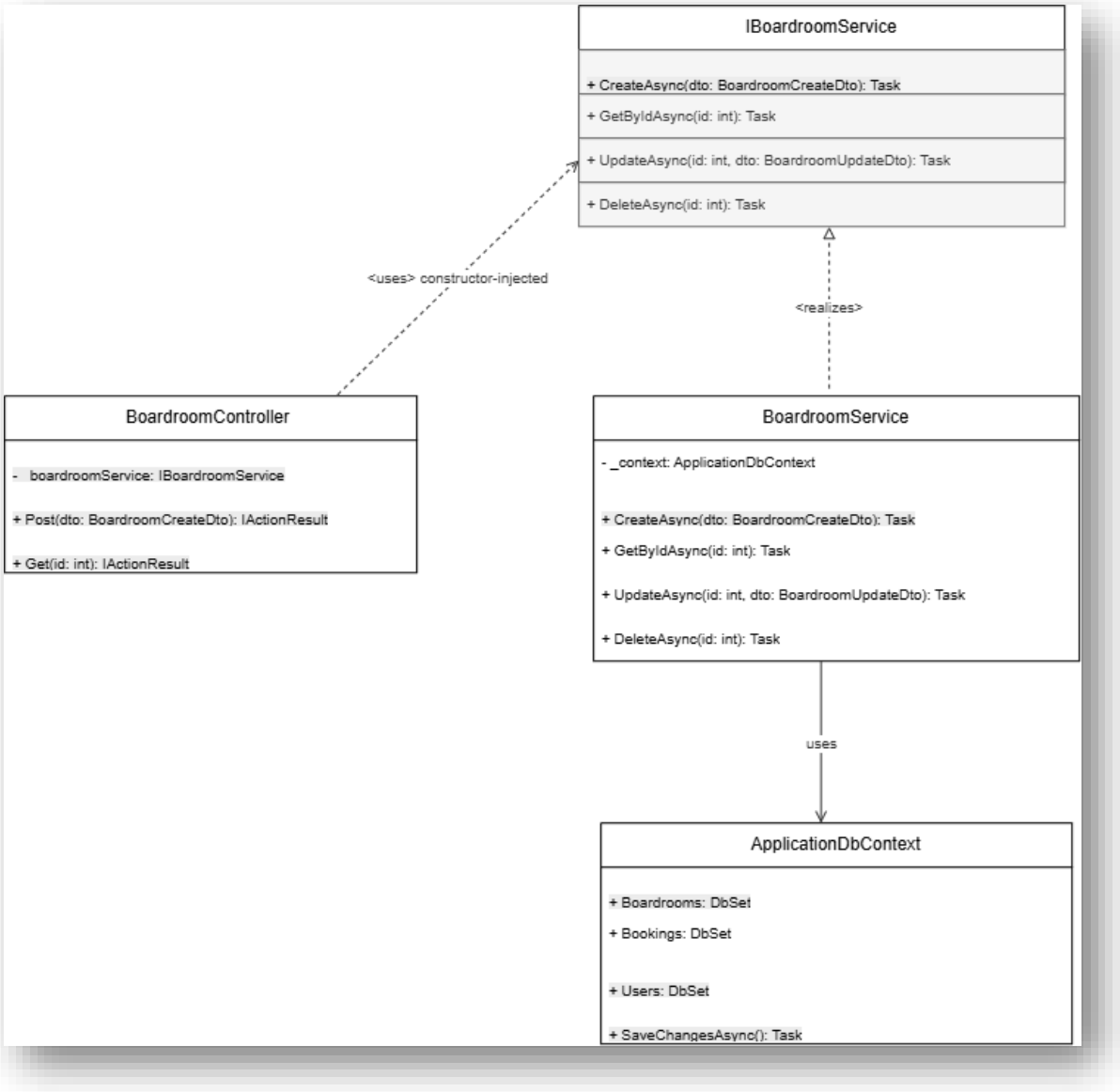


Figure 9:UML diagram of Boardroom implementation.

How the diagram demonstrates the Service layer and dependency injection

The sequence diagram above shows what happens when a POST /api/Boardroom request is made from start to finish. The **BoardroomController → IboardroomService** part shows **Dependency Injection** because the controller works with the IBoardroomService interface instead of directly using the actual service class.

The **Service Layer Pattern** can be seen in the **BoardroomService** section. This is where the main work happens, such as mapping the DTO to an entity, using **ApplicationDbContext** to save the data, and mapping the result back into a DTO. The controller does not need to know how all of this happens**; it simply sends the request to the service and receives the result back.**

9.1.12.2 Architecture Patterns

Architecture Pattern: Layered Architecture

FlexiSpace uses a **Layered Architecture** to separate the different responsibilities of the system. The main layers are the Core, API, and Infrastructure layers, with each layer handling a specific part of the application.

FlexiSpace.API: Is responsible for handling HTTP requests and responses. Depends on Core directly (for the interfaces it calls) and on Infrastructure only at startup, to wire the concrete implementations in. So, this layer contains the controllers that expose the API endpoints to the web and mobile applications.

FlexiSpace.Core contains the main parts of the application which includes entities, DTOss, interfaces (IboardroomService, IBookingService, etc.), Enums and service contracts. This layer contains the definitions that are then shared across the system without being directly tied to the database. It has zero dependencies on any other project, so it does not know EF Core exists, or that it's being served over HTTP.

FlexiSpace.Infrastructure: It implements Core's interfaces (e.g. BoardroomService, BookingService) and also handles external resources. So, this includes Entity Framework Core, ApplicationDbContext, database configurations and Microsoft Entra ID/Graph integration.

The database is accessed through **Entity Framework Core** and stores the FlexiSpace data in Azure SQL Database.

Why we chose this architecture.

We chose Layered Architecture because it gives each part of FlexiSpace a clear responsibility. For example, a BookingController does not directly communicate with the database. Instead, it passes the request to the appropriate service, which will then use Entity Framework Core to access the database.

So, this separation makes the system easier to **maintain test** and **Scale**. For example, if we need to change how the database is accessed, the changes can mainly be made in the infrastructure layer without changing the API controllers.

It also supports our requirement for **one shared backend**. Both the web application and mobile application can communicate with the same API instead of having separate business logic for each application. This matches the project's planned technology stack of React, NET 8 Web API, Microsoft Graph, SQL Server, and Microsoft Azure.

System Architecture Diagram: Layered Architecture

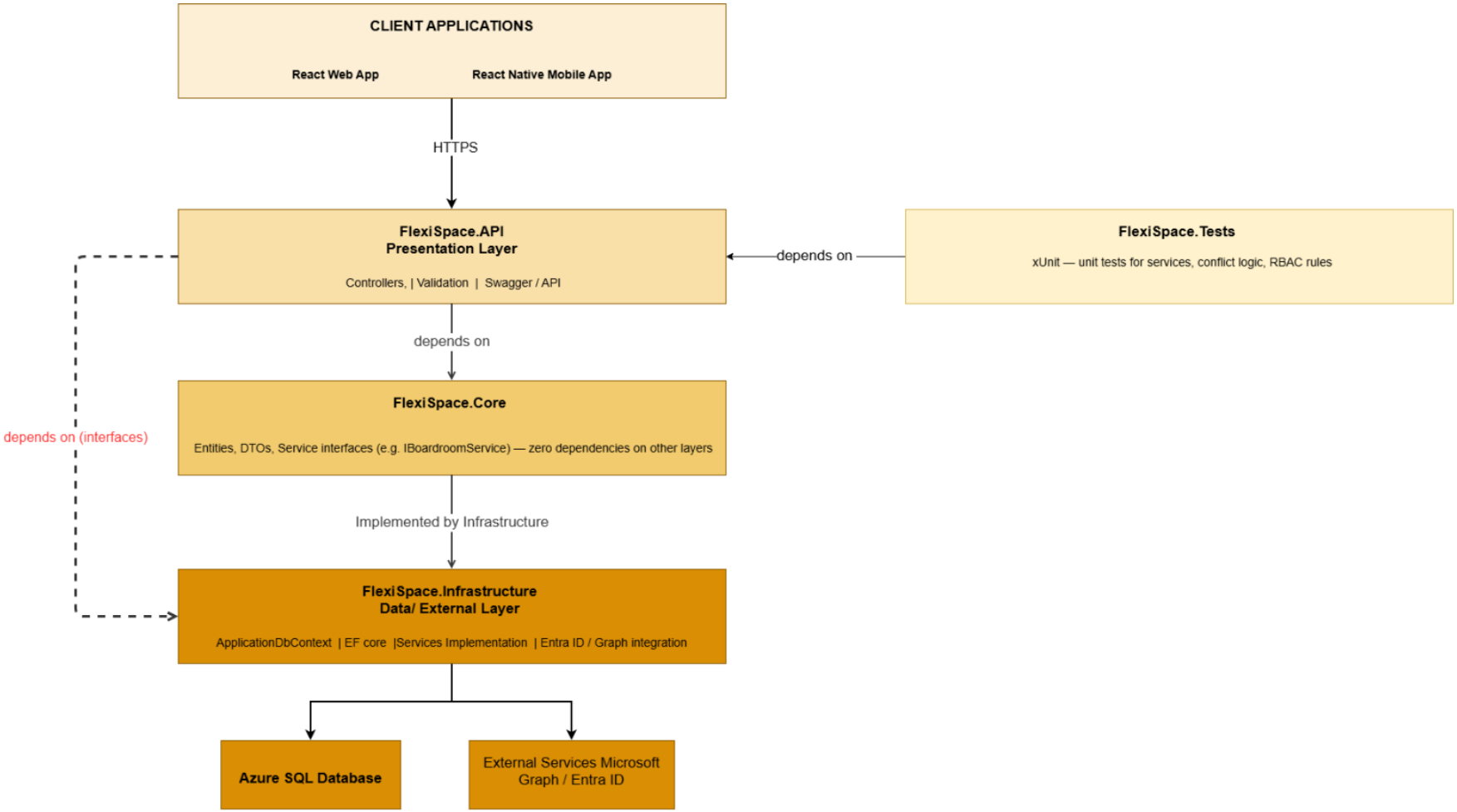


Figure 10: FlexiSpace Layered Architecture

How the diagram demonstrates the architecture

So, the diagram shows how a request moves through the different layers of FlexiSpace. The client applications communicate with the API layer through HTTPS. The API controllers then use interfaces from the Care layer, while Infrastructure layer provides the actual implementations and handles database access through Entity Framework Core.

This means that each layer has its own responsibility instead of putting everything into the controllers, which would make it hard to scale and maintain. It also allows the same backend to be used by both the web and mobile applications.

9.1.12.3 Cloud Architecture

FlexiSpace uses **Microsoft Azure** to host and support the main services required by the system. The architecture connects the web and mobile applications to the shared ASP.NET Core Web API, which then communicates with Azure SQL Database and Microsoft services.

Main Components

Component	Purpose in FlexiSpace
React Web Application	Provides the web interface used by users and managers.
React Native Mobile Application	Provides the mobile interface for users.
ASP.NET Core Web API	Acts as the central backend and provides the REST API used by both applications.
Azure SQL Database	Stores users, locations, boardrooms, bookings, equipment, catering, and other system data.
Microsoft Entra ID	Handles user authentication and provides access tokens for protected API requests
Microsoft Graph	Allows the system to interact with Outlook calendars for booking synchronisation

Communication and Protocols

- **HTTPS/TLS:** Used to securely communicate between the client applications and API.
- **REST/HTTPS:** The React web and mobile applications communicate with the ASP.NET Core API through REST endpoints.
- **OAuth 2.0/ OpenID Connect:** Used with Microsoft Entra ID for authentication and obtaining access tokens.

- **Bearer Access Tokens:** The client sends the access token with protected API requests so that the API can authenticate and authorise the user.
- **Microsoft Graph API:** Used by the backend to communicate with Outlook calendars.

Why We Chose This Architecture

- **One shared backend –** both the mobile and website application use the same API and business logic.
- **Integration with the client's existing environment** - Microsoft Entra ID and Outlook calendars can be integrated through Microsoft's services since they already have Microsoft accounts.
- **Security –** HTTPS and Entra ID protect communication and access to the API.
- **Centralised data –** users, bookings, boardrooms, and other information are stored in on database.
- **Scalability -** additional users, boardrooms and locations can be added without creating separate systems as FlexiSpace does intend to add more locations which means more boardrooms as well, so the system is created to easily add more locations.

Network flow Explanation:

The diagram below shows how a request moves through the system:

1. Authentication: Before a user can access protected FlexiSpace features:
   - The **React Web App** (browser) and **.NET MAUI Mobile App** each redirect the user to **Microsoft Entra ID** over HTTPS using OAuth 2.0 / OIDC (web via standard redirect flow, mobile via MSAL — Microsoft Authentication Library).
   - After the user is authenticated, Entra ID provides an **access token**, which includes RBAC role claims (e.g. Centre Manager, Staff, Admin) which will be used later for authorization.
2. Client to API
   - After authentication, the web or mobile application sends requests to the **ASP.NET Core Web API** hosted on Azure App Service.
   - These requests use **HTTPS** and the access token is included as a Bearer token.
   - The API then checks the user's authentication and authorisation before allowing access to protected resources.
3. API to Database
   - Once the request is authorised, the API processes the request through the relevant service.

- **Entity Framework Core** is used by FlexiSpace.Infrastructure to communicate with **Azure SQL Database**.
- The database connection uses an encrypted connection to protect the data being transferred.
- For example, when a user creates a booking, the booking information is processed by the API and stored in Azure SQL Database.

4. API to Microsoft Graph
    - When calendar functionality is required, the API communicates with Microsoft Graph API over HTTPS.
    - Microsoft Graph allows FlexiSpace to work with the client's Outlook calendars.
    - This supports the requirement for calendar synchronisation and helps keep bookings aligned with the existing calendar system.
5. Frontend Delivery
    - Azure Static Web Apps provides the compiled React application to the user's browser.
    - Once the application has loaded, the frontend communicates with the API for operations such as viewing availability and creating bookings.

Architecture Diagram of FlexiSpace

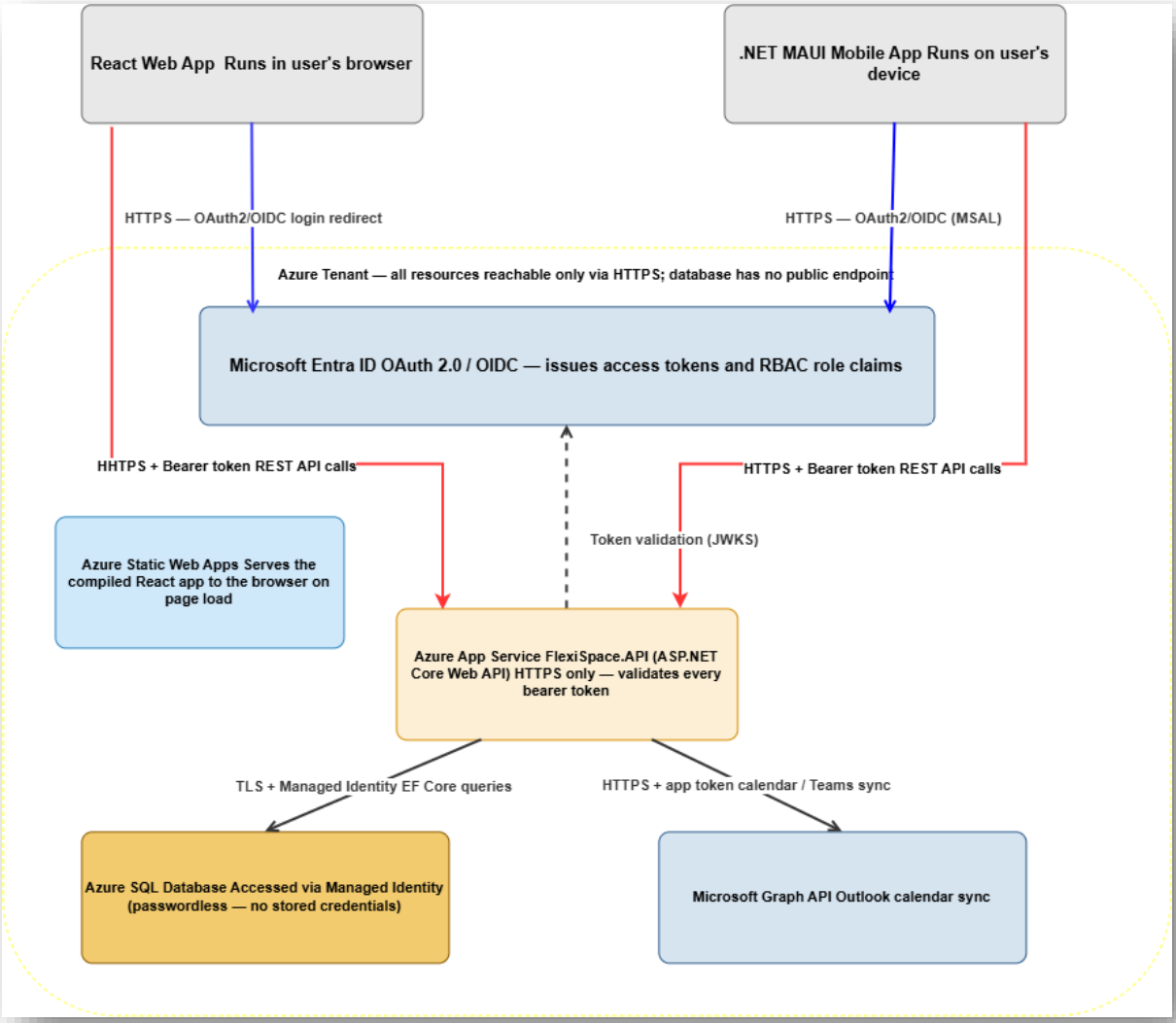


Figure 11:Cloud Architecture diagram.

9.1.13 Security

Security was considered throughout the design of FlexiSpace because the system will handle personal information, booking information and access to Microsoft 365 calendars. Since FlexiSpace will have both a **mobile application and a website**, security will be applied across the applications, API, and database, with the **mobile application being the main focus**.

The security approach will be guided by **OWASP**, particularly its mobile application security guidance and common web application security risks. OWASP Mobile application Security Verification Standard (MASVS) is being used as a guide to identify possible vulnerabilities rather than being the only basis for the security design (Project), 2024).

9.1.13.1 Microsoft Entra ID Authentication

Security used

FlexiSpace will use **Microsoft Entra ID** for user authentication which will be applied to. NET MAUL mobile application, React website and FlexiSpace API.

- Users will sign in using their Microsoft account.
- OAuth 2.0 / OpenID Connect will be used for the authentication process.
- The mobile application and website will use the same identity system.
- After successful authentication, Entra ID provides an access token.
- The access token will be sent with requests to protect API endpoints.
- The API will validate the token before allowing access to protected resources.
- Passwords will not be stored in the FlexiSpace database because authentication is handled by Microsoft Entra ID.

Why it is used:

This protects the system against risks such as unauthorized access and authentication attacks while also fitting the client's Microsoft environment. The project plan for FLexiSpace specifies Microsoft 365 login and Microsoft Entra ID as part of the system design.

Threats it protects against:

- Unauthorised access
- Password exposure

- Weak password management
- Credential theft

9.1.13.2 Authorisation and RBAC (Role based Access control)

Security used

FlexiSpace will use Role-Based Access Control to determine what each user is allowed to access. The permission checks will happen I the backend API not just in the mobile or web interface.

FlexiSpace will use RBAC for its different user types.

Role	Example permissions
Administrator	Manage users, locations, boardrooms, and system settings
Centre Manager	Manage bookings and boardrooms for their location.
Staff	View availability and create/manage their bookings
Client	Make bookings and view relevant booking information

Why it is used.

Simply knowing that someone is logged in is not enough, different users have different responsibilities. For example, a Staff member should be able to make a booking but should not automatically be allowed to manage users or system configuration.

Threats it protects against

- **Broken Access Control**, which OWASP identifies as one of the major web application security risks (OWASP, 2025).
- Unauthorised functionality
- Users accessing another role's information.
- Privilege escalation

9.1.13.3 Secure Token Storage on the Mobile Device

Security used

FlexiSpace will use token-based authentication to make sure that requests to protected API endpoints come from authenticated users. This will be applied to the Mobile application API, Website API, and API authentication middleware.

The access token received from Microsoft Entra ID will be sent with API requests using the Authorization: Bearer header.

When the API receives a request, it will not simply trust the token. It will validate information such as:

- Who issued the token.
- Whether the token is intended for the FlexiSpace API.
- Whether the token has expired.
- Whether the token is valid.
- What claims and permissions are associated with the user.

Only after the token has passed validation will the request continue into the API's controller and service layers.

Why It Is Used

The API needs to verify who is making a request before providing or changing protected information.

Threats It Protects Against

- Forged tokens
- Expired tokens
- Unauthenticated requests
- Misuse of stolen authentication tokens

FlexiSpace example: If someone attempts to access GET /api/Boking without a valid access token, the API rejects the request instead of returning booking information.

9.1.13.4 Secure Storage on the Mobile Application

Security Used

Sensitive authentication information used by the mobile application will be stored using **secure device storage** rather than ordinary text files or easily accessible application preferences. This will be applied to .NET MAUI mobile application and user's mobile device.

The mobile application will also avoid storing information that it does not need.

For example, the application should not contain:

- User passwords

- Database credentials
- Microsoft Graph secrets
- API keys
- A complete copy of the booking database

Instead, the application retrieves the information it needs through the API.

Why It is Used

Mobile devices can be lost, stolen, or compromised. Storing sensitive information insecurely increases the damage that could occur if someone gains access to the device.

Threats It Protects Against

- Token theft
- Credential exposure
- Sensitive data exposure
- Compromised or stolen devices

This follows the OWASP MASVS focus on secure storage and protecting sensitive information within mobile applications.

FlexiSpace Example: If a Staff member loses their phone, someone should not be able to open a normal text file on the device and retrieve their FlexiSpace authentication information.

9.1.13.5 HTTPS/TLS Network Security

Security used

FlexiSpace will use **HTTPS/TLS** to encrypt communication between the different components of the system. It will apply to the Mobile app, website, API, Azure SQL connection, and Microsoft Graph communication.

The main communication paths are:

Mobile App — HTTPS → FlexiSpace API

Website ——— HTTPS → FlexiSpace API

API ————— TLS →Azure SQL

API ————— HTTPS →Microsoft Graph

This means information such as access tokens, booking details and personal information is encrypted while travelling between systems.

Why it is used:

Without secure communication, someone monitoring the network could potentially intercept information being sent between the user's device and FlexiSpace**.**

Threats It Protects Against

- Man-in-the-middle attacks.
- Network sniffing
- Token interception
- Data interception

FlexiSpace example: A Staff member creates a booking while using public Wi-Fi. HTTPS protects the booking request and access token while they travel from the mobile application to the FlexiSpace API.

9.1.13.6 API as the Security Boundary

Security Used

The **FlexiSpace API acts as the security boundary** between the client applications and the database.

The mobile application and website will not connect directly to Azure SQL.

Instead:

Mobile Application → FlexiSpace. API → Authentication → Authorisation → Input Validation → Service Layer → Entity Framework Core → Azure SQL

Every request therefore passes through the API before database information is accessed.

Why It Is Used

This gives FlexiSpace one controlled point where security checks can be performed. It also prevents database credentials from being placed inside the mobile application or website.

Threats It Protects Against

- Direct database attacks
- Database credential exposure
- Bypassing API security
- Unauthorised database access

9.1.13.7 Server-Side Input Validation

Security Used

FlexiSpace will validate information **on the server**, even when the mobile application and website also perform validation. This will be applied to the API controllers, DTOs, Service layer, and Booking logic.

The API will treat information received from the client as untrusted until it has been checked.

For a booking, the API will check things such as:

- Required fields are provided.
- The boardroom exists.
- The date and time are valid.
- The booking is not in the past.
- The start time is before the end time.
- The number of attendees does not exceed room capacity.
- Equipment and catering options exist.
- The requested time does not conflict with another booking.

Why It Is Used

Frontend validation can be bypassed. Someone could modify the application or use Postman to send their own request directly to the API. Therefore, the backend must perform the final validation.

Threats It Protects Against

- Malicious input
- Manipulated requests.
- Invalid bookings
- Data corruption
- Bypassed frontend validation

FlexiSpace Example: The mobile app might prevent a user from selecting 50 attendees for a 10-person boardroom. So, if the user bypasses the app and sends 50 directly to the API, the API still rejects the booking.

9.1.13.8 Secrets Management

Security Used

Sensitive configuration such as database credentials and API keys will be stored using **User Secrets, environment variables, and secure Azure configuration**, rather than being written directly into the source code.

Why It Is Used

So, since the source code is stored in GitHub and shared between team members, putting real credentials inside the source code could expose them to anyone who gains access to the repository.

Threats It Protects Against

- Credential leakage
- Database compromise
- API key exposure
- Accidental GitHub exposure

FlexiSpace Example: The Azure SQL connection string is stored as secure configuration instead of being written directly into appSettings.

9.1.13.9 Cross-Site Scripting (XXS)

Security Used

For the React website, FlexiSpace will rely on **React's normal output escaping**, combined with server-side validation and sanitisation where user-controlled HTML needs to be handled (Project), n.d.).

Why It Is Used

XSS could allow an attacker to insert malicious JavaScript into information that is later displayed to another user.

Where It Is Applied

Mainly:

- React website.
- API input validation
- User-controlled content such as booking notes.

Threats It Protects Against

- Cross-Site Scripting

- Malicious JavaScript execution
- Session/token theft through injected scripts
- Manipulation of the user's browser

9.1.13.10 Audit Logging

Security Used

FlexiSpace will use an **AuditLog** to automatically record important actions performed in the system.

The audit record can contain:

- User who performed the action
- Action performed
- Entity affected
- Entity ID
- Date and time.
- Previous value
- New value

So, the AuditLog is system-managed rather than being exposed as normal CRUD functionality.

Why it is used:

Security is not only about preventing an attack. If something goes wrong, we also need to be able to investigate **what happened and who performed the action**.

Threats It Protects Against

- Unauthorised changes
- Suspicious activity
- Lack of accountability
- Difficulty investigating incidents.

9.1.14 DevOps

9.1.14. 1 Repository Structure

The GitHub repository will be organized into the following top-level folders:

FlexiSpace-Booking-System

```
├── Backend
│   ├── Data            # Database schema, EF Core models, migrations (Khensani)
│   ├── Bookings        # Booking CRUD, conflict detection logic (Tino)
│   └── Users           # RBAC, admin endpoints, audit trail, notifications (Denzel)
├── Website             # React + Fluent UI application (Khumo)
├── Mobile              # .NET MAUI application (Riba)
├── Documentation       # SRS, user guides, architecture diagrams
└── README.md           # Setup instructions for all components
```

9.1.14.2 Branching Strategy

We will adopt a feature-branch workflow with the following conventions:

- **Main Branch (**main**):** The production-ready branch.
- **Feature Branches (**feature/**):** Created from main for each task or issue.
    - Naming format: feature/short-description (e.g., feature/booking-conflict-check, feature/audit-log-endpoint)
- **One Branch Per Issue:** Each feature branch must be associated with a single GitHub Issue.
- **Bugfix Branches (**bugfix/**):** For emergency fixes to production.

9.1.14.3 Pull Request Process

All changes must be merged via a Pull Request (PR) with the following process:

1. Developer creates a feature branch and commits their changes.
2. Developer opens a Pull Request targeting main.
3. At least one teammate must review the PR and approve it.

4. All automated checks (build, unit tests) must pass.
5. The PR is merged by the reviewer.
6. The branch is deleted after successful merge.

9.1.14.4 Commit Messages

Commit messages must reference the issue number for traceability:

feature/booking-conflict-check #42 - Implemented conflict detection algorithm

9.14.5 Project Management (GitHub Projects)

We will use GitHub Projects to manage our workflow with the following columns:

Column	Description
To Do	Issues that have been created but not yet started
In Progress	Issues currently being worked on (assigned to one person)
In Review	Issues awaiting PR review
Done	Issues completed and merged

Issues: Every task is a GitHub Issue with:

- One responsible assignee
- Appropriate labels (backend, website, mobile, documentation)
- Milestone association (Phase 1, Phase 2, Phase 3)

Milestones: We will use milestones matching the project phases:

- Phase 1: Documentation & Prototype (Due 17 Aug)
- Phase 2: Build (Due 28 Sept)
- Phase 3: Final Submission (Exam Week 1)

Critical: Real secrets (connection strings, client secrets, API keys) must never be committed to the repository.

9.1.14.6 CI/CD Pipeline (Plan)

The following automation will be implemented for the Azure-hosted solution:

- **Continuous Integration (CI):** On every PR to main, automatically run:
    - Build for Backend, Website, and Mobile
    - Unit tests for booking logic and RBAC
    - Code style and linting checks
- **Continuous Deployment (CD):** On successful merge to main, automatically deploy:
    - Backend API to Azure App Service
    - Database migrations to Azure SQL Database (planned for Phase 2

9.1.14.7 Attendance Tracking

Khensani (team leader) will log weekly lecturer session attendance in the repository,

9.1.14.8 README Maintenance

The README.md file must be kept current with setup steps for all three components (Backend, Website, Mobile) as they are built. Each component's setup section must include:

- Prerequisites (software required)
- Installation steps
- Environment configuration instructions
- Running the application locally

9.1.15 Running Costs

9.1.15.1 Purpose

This document explains the ongoing monthly costs of running the FlexiSpace boardroom booking system once it is live, based on the technology stack in the Project Plan (React front end, .NET 8 Web API, Microsoft SQL Server, Microsoft Graph API, Microsoft Entra ID, hosted on Microsoft Azure). It is intended to support the Code Unit Proposal Presentation and give the team a shared, plain-language reference for explaining these numbers to stakeholders.

9.1.15.2 Running Costs vs. Build Costs

Everything described in the Project Plan — development of the React front end, the .NET 8 API, and the Graph API integration — is a one-time build cost covered by the development team's time. "Running costs" are separate: they are what FlexiSpace pays Microsoft every month, indefinitely, simply to keep the system hosted, storing data, and synchronised with Outlook. This document covers running costs only.

9.1.15.3 Cost Components Explained

3.1 App Service — hosting the application

Azure App Service is where the React front end and .NET 8 Web API actually run. Azure manages the underlying server: patching, restarts, and providing a public URL. The cost is based on compute hours — how much CPU and memory is reserved for the app, and for how long.

- Basic tier (~$13-15/month): a single fixed-size instance with no auto-scaling. Suitable for a pilot across the three scoped locations, where traffic is light.
- Standard tier (~$57+/month): adds auto-scaling, so if traffic spikes (for example, everyone booking rooms on a Monday morning), Azure automatically adds a second instance instead of the app slowing down.
- Linux is used instead of Windows because .NET 8 runs natively on Linux, avoiding the extra Windows Server licensing fee bundled into Windows-based App Service plans.

Team takeaway: Start on Basic to pilot and demo the system; move to Standard once real clients are booking rooms and downtime has a real business cost.

3.2 Azure SQL Database — storing bookings, users, and rooms

This holds every table described in the Project Plan's database design: Bookings, Boardrooms, Users, Equipment, and Locations. Azure SQL's DTU model bundles CPU,

memory, and I/O into a single tier, which keeps pricing simple to reason about for a system of this size.

- Basic (~$5/month): 5 DTUs, 2GB storage — sufficient for early testing.
- Standard S0/S1 (~$15-30/month): more headroom, and realistically where a three-location booking system with a modest number of monthly bookings would sit comfortably.

Team takeaway: A booking system is not data-intensive — it stores structured rows of bookings, not images or video — so this component stays cheap even as usage grows.

3.3 Microsoft Entra ID — login and authentication

Entra ID (formerly Azure Active Directory) is Microsoft's identity service. It lets staff log in with their existing FlexiSpace Microsoft 365 account instead of the team building a separate login system from scratch.

- Free tier: covers standard single sign-on, which is likely sufficient for an internal, staff-facing tool.
- Entra ID P1 (~$6/user/month): optional — only needed for extras such as conditional access policies (e.g. blocking logins from outside South Africa). Worth treating as a future upgrade rather than a required MVP cost.

Team takeaway: Because FlexiSpace already has Microsoft 365 for the Centre Managers' Outlook calendars, this identity system is being reused, not paid for twice.

3.4 Microsoft Graph API — the Outlook sync

Graph API is the mechanism behind the system's core promise of no double bookings — it is what reads from and writes to the Centre Managers' Outlook calendars (centurion@, rebecca@, antoinette@, lesego@).

- Cost: $0. Graph API calls are not billed separately. The only cost driver would be a brand-new mailbox needing a new Microsoft 365 licence — but all four mailboxes already exist.

Team takeaway: The most technically complex integration in the system is also, from a running-cost perspective, free.

3.5 Storage, monitoring, and SSL — the small items

- Blob storage (~$1-2/month): for exported reports or logs — negligible at this scale.
- Application Insights (free up to 5GB/month): Azure's monitoring tool, flags slowness or errors. The free tier covers a small application easily.
- SSL certificate: free, via App Service's managed certificate for the custom domain.

9.1.15.4 Two Cost Scenarios

Running cost depends on how heavily the system is used, so the team should present a range rather than a single number.

Scenario A — Pilot / MVP (3 locations as scoped, ~15 boardrooms)

Component	Tier / Notes	Est. Monthly (USD)	Est. Monthly (ZAR)
App Service (React + .NET 8 API)	Linux, Basic B1	$13 - $15	R210 - R245
Azure SQL Database	Basic / Standard S0	$5 - $15	R80 - R245
Microsoft Entra ID	Free tier (covered by existing M365)	$0	R0
Microsoft Graph API calls	Usage-based, no separate charge	$0	R0
Blob storage (logs, attachments)	Pay-as-you-go	$1 - $2	R15 - R30
Application Insights	Free tier (5GB/month)	$0	R0
SSL certificate	Included (App Service managed cert)	$0	R0
Total (approx.)		**$20 - $35**	**R325 - R570**

Scenario B — Production / Scaled (auto-scaling, more locations, reliability SLA)

Component	Tier / Notes	Est. Monthly (USD)	Est. Monthly (ZAR)
App Service	Linux Standard S1, auto-scale (1-2 instances)	$57 - $115	R925 - R1,865
Azure SQL Database	Standard S1 - S2	$30 - $75	R485 - R1,215
Entra ID P1 (optional)	Conditional access / SSO extras, per user	$6 / user	R95 / user
Storage + backups	Pay-as-you-go	$5 - $10	R80 - R160
Application Insights (extended)	Beyond free tier	$5 - $15	R80 - R245
Bandwidth / egress	Pay-as-you-go	$5 - $10	R80 - R160
Total (approx.)		**$150 - $300**	**R2,400 - R4,900**

9.1.16 Change Management

Getting people to actually use the new system is just as important as building it right. Below is the plan for that.

9.1.16.1 Getting the Organisation On Board

The company will want this system because it fixes a real headache — double-bookings that frustrate customers and waste staff time. It will save Centre Managers hours each week, make customers happier with 24/7 self-service booking, and give management data on room usage they never had before.

9.1.16.2 Getting Users On Board

- **Centre Managers:** Will value being able to see all bookings for their location in one place, without having to manually update Outlook.
- **Staff:** Will appreciate how quick and easy it is to book a room without checking with a Centre Manager first, plus instant confirmations and reminders.

Adoption Plan

- **Early Announcements:** The new system will be communicated well before launch through emails, intranet posts, and team meetings.
- **Stakeholder Involvement:** Centre Managers will help test the system. When people feel they contributed, they become advocates.
- **Executive Support:** The CEO publicly backing the project gives it weight.
- **Champion Network:** Tech-savvy staff will become "Champions" who help their colleagues.

Training

- Video tutorials for common tasks
- Live webinars for Q&A
- One-on-one help for those who need it
- Quick reference guides with screenshots

9.1.16.3 Handing Over to the Client

Milestone	Activity
Two Weeks Before	Final testing signed off.
One Week Before	Knowledge transfer session with IT team.
Go-Live Day	Project team on standby for deployment.
First Two Weeks	"Hyper-care" support period.

Milestone	Activity
After That	FlexiSpace IT takes over day-to-day issues.

9.1.16.4 Support Plan

All issues are tracked on GitHub. A User Guide helps people solve common problems. If the system goes down, a documented fallback plan is in place.

9.1.16.5 Transitioning from Old to New

- **Parallel Running:** For a few weeks, the old process remains available as a backup.
- **Data Migration:** All existing bookings are moved before launch.
- **Post-Transition:** FlexiSpace IT takes over. The project team stays available for complex issues.

9.1.17 Appendices

Declaration of authenticity

Declaration of authenticity

I, __Khensani Maluleke__ ID Number, __0502220318088__ hereby declare that this portfolio, and any evidence included therein, contains my own independent work and that I have not received help from other groups.

I confirm that we have not committed plagiarism in the accomplishment of this work, nor have I falsified and/or invented experimental data.

I accept the academic penalties that may be imposed for violations of the above.

STUDENT NUMBER: ____ST10451309__

STUDENT NAME: _____Khensani Maluleke___

STUDENT SIGNATURE: _____ DATE: ___2026/08/11____



References

Elmasri, R. & Navathe, S. B., 2016. *Fundamentals of Database Systems.* 7th ed. Harlow(UK): Pearson.

Microsoft, 2026. *Dependency injection in ASP.NET Core.* [Online]
Available at: https://learn.microsoft.com/en-us/aspnet/core/fundamentals/dependency-injection
[Accessed 11 August 2026].

OWASP, 2025. *OWASP Top 10:2025.* [Online]
Available at: https://owasp.org/Top10/2025/

Project), O. (. W. A. S., 2024. *Mobile Application Security Verification Standard (MASVS).* [Online]
Available at: https://mas.owasp.org/MASVS/
[Accessed 11 August 2026].

Project), O. (. W. A. S., n.d. *Cross Site Scripting Prvention Cheat Sheet.* [Online]
Available at: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html
[Accessed 11 August 2026].

Fowler, M., 2002. Patterns of Enterprise Application Architecture. Boston: Addison-Wesley.

GitHub, 2026. GitHub Actions documentation. [online] Available at: https://docs.github.com/en/actions [Accessed 16 August 2026].

Kotter, J.P., 1996. Leading Change. Boston: Harvard Business School Press.

Microsoft, 2026a. API support in Azure Static Web Apps with Azure App Service. [online] Microsoft Learn. Available at: https://learn.microsoft.com/en-us/azure/static-web-apps/apis-app-service [Accessed 16 August 2026].

Microsoft, 2026b. Designing the infrastructure persistence layer. [online] Microsoft Learn. Available at: https://learn.microsoft.com/en-us/dotnet/architecture/microservices/microservice-ddd-cqrs-patterns/infrastructure-persistence-layer-design [Accessed 16 August 2026].

Microsoft, 2026c. Outlook calendar API overview - Microsoft Graph. [online] Microsoft Learn. Available at: https://learn.microsoft.com/en-us/graph/outlook-calendar-concept-overview [Accessed 16 August 2026].

Microsoft, 2026d. What is Azure Static Web Apps? [online] Microsoft Learn. Available at: https://learn.microsoft.com/en-us/azure/static-web-apps/overview [Accessed 16 August 2026].

Object Management Group, 2017. About the Unified Modeling Language Specification Version 2.5.1. [online] Available at: https://www.omg.org/spec/UML/2.5.1/About-UML [Accessed 16 August 2026].

Prosci, 2024. Kotter's Change Management Theory: Explanation and Applications. [online] Available at: https://www.prosci.com/blog/kotters-change-management-theory [Accessed 16 August 2026].

Anthropic (2026) Claude (Sonnet 4.6) [Large language model]. Available at: https://claude.ai (Accessed: August 2026).

Cloudwards (2026) Microsoft Azure Pricing in 2026 [Understanding Cloud Prices]. Available at: https://www.cloudwards.net/azure-pricing/ (Accessed: August 2026).

Kuberns (2026) Azure Pricing 2026: Real Costs, Plans & Cheaper Options. Available at: https://kuberns.com/blogs/azure-pricing/ (Accessed: August 2026).

Pump.co (2025) Azure SQL Database Pricing: What You Need to Know. Available at: https://www.pump.co/blog/azure-sql-database-pricing/ (Accessed: August 2026).

Synextra (2026) Azure SQL Pricing: How It Works And How To Reduce Costs. Available at: https://www.synextra.co.uk/knowledge-base/azure-sql-pricing/ (Accessed: August 2026).

Xe (2026) 1 USD to ZAR - US Dollars to South African Rand Exchange Rate. Available at: https://www.xe.com/en-us/currencyconverter/convert/?Amount=1&From=USD&To=ZAR (Accessed: August 2026).